

Maschinelles Lernen

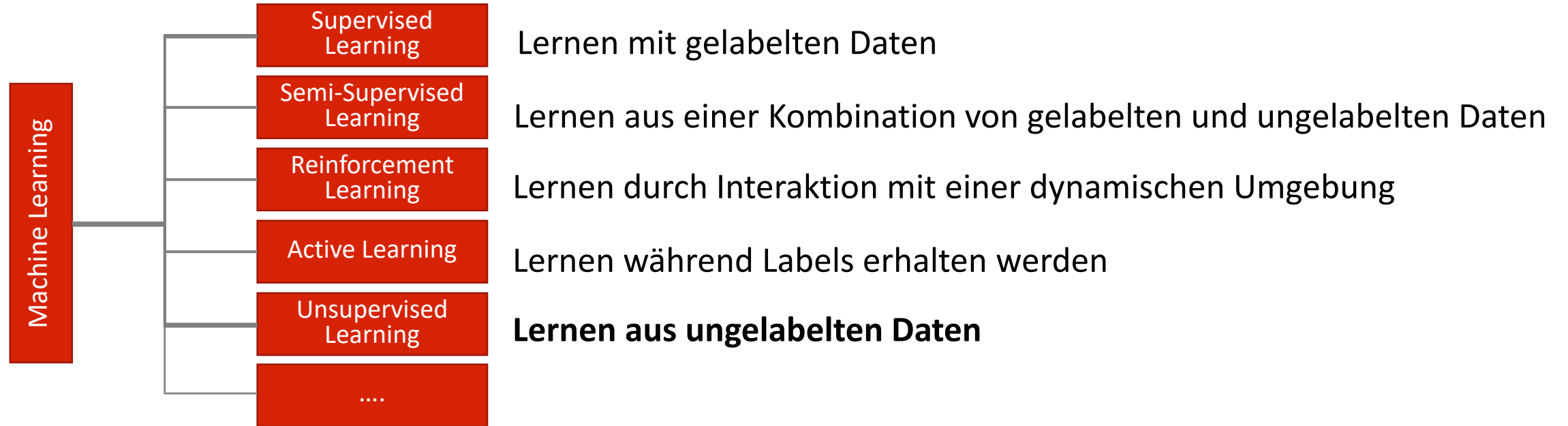


Prof. Dr. Christine Preisach

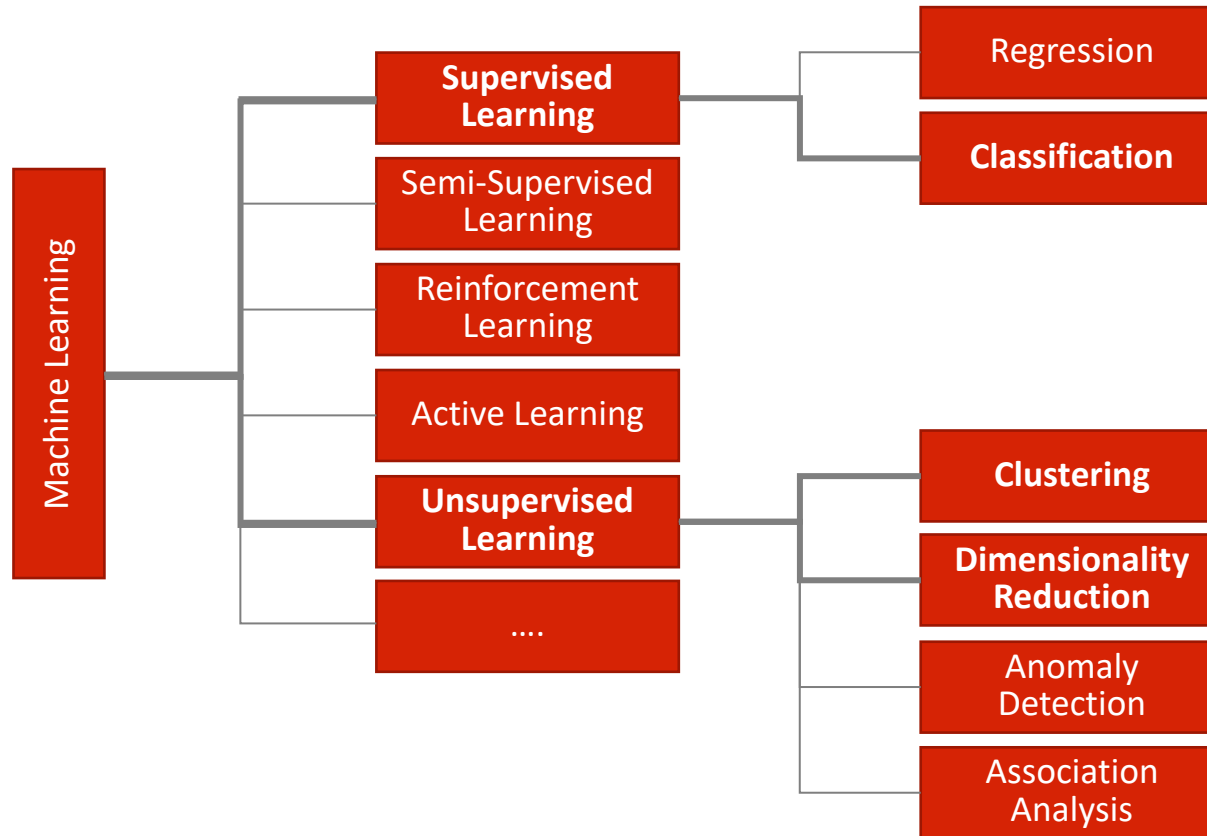




Kategorien von ML Lernansätzen

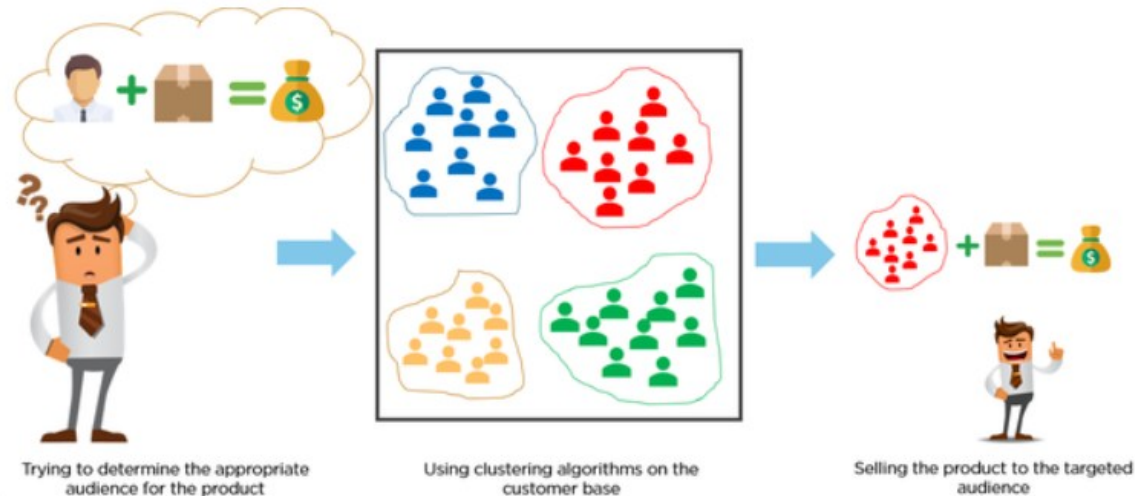


ML Problemstellungen (in dieser Vorlesung)



Beispielanwendungen - Clustering

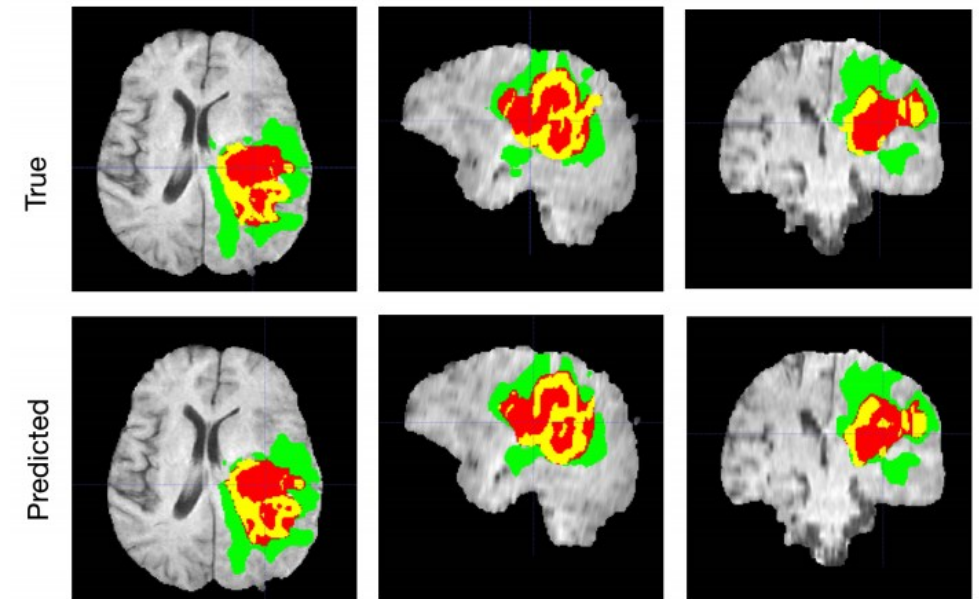
- **Marktsegmentierung:** Angenommen, wir haben Zugriff auf eine große Anzahl von Daten (z. B. mittleres Haushaltseinkommen, Beruf, Entfernung zum nächsten städtischen Gebiet usw.) für eine große Anzahl von Menschen.
- Unser Ziel ist es, eine Marktsegmentierung durchzuführen, indem wir **Untergruppen** von Menschen identifizieren, die möglicherweise empfänglicher für eine bestimmte Werbeform sind oder mit größerer Wahrscheinlichkeit ein bestimmtes Produkt kaufen. Die Aufgabe der Marktsegmentierung besteht darin, die Personen im Datensatz zu **gruppieren**.



[Customer segmentation with clustering - medium.com](https://medium.com/customer-segmentation-with-clustering)

Bildsegmentierung:

- Die Bildsegmentierung ist ein wesentlicher Bestandteil der Bildverarbeitung. Die Bildsegmentierung kann durch Clustering erreicht werden.
- Die Magnetresonanztomographie (MRT) ermöglicht eine Visualisierung der inneren Strukturen von Objekten und lebenden Organismen. Die Segmentierung eines MRT-Bildes ist eine Schlüsselaufgabe in vielen medizinischen Anwendungen, beispielsweise bei der chirurgischen Planung und der Erkennung von Anomalien.
- Die MRT-Segmentierung zielt darauf ab, ein Eingabebild in wichtige anatomische Bereiche zu unterteilen, die jeweils hinsichtlich bestimmter Bildeigenschaften einheitlich sind.
- Die MRT-Segmentierung kann als Clustering-Problem formuliert werden, bei dem ein Satz von Merkmalsvektoren (erhalten durch Transformation), die Bildmessungen und Pixelpositionen enthalten in eine Reihe von Strukturen gruppiert wird.



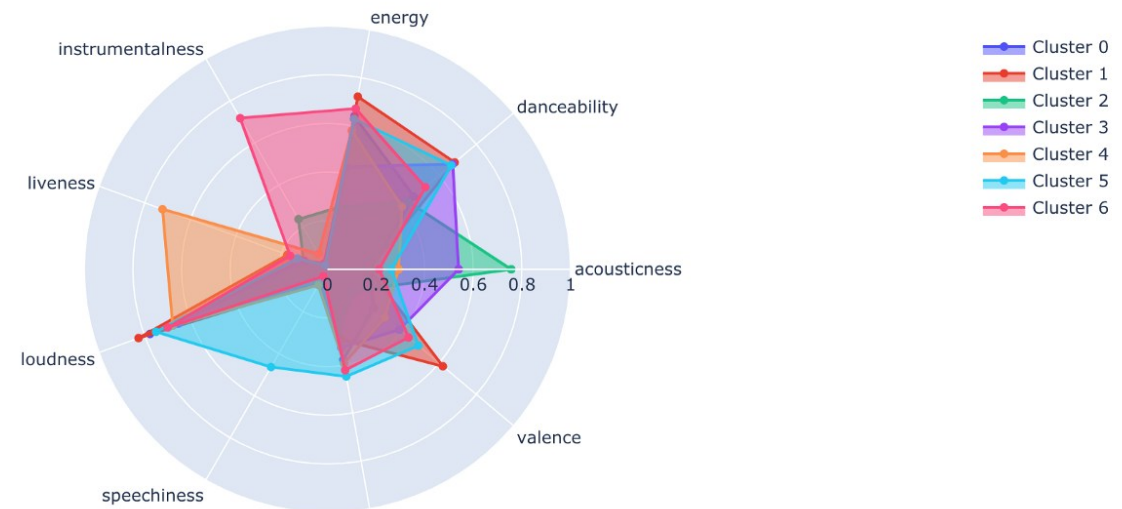
Beispiele: Musiksegmentierung

Annahme: Musik wird in Kategorien unterteilt, und Kunden bevorzugen einige Kategorien

Nutzerbasiert: Ein Song wird durch die Nutzer die ihn gehört haben repräsentiert. Annahme ähnliche Musik gehört zu ähnlichen Kundengruppen und umgekehrt

Inhaltsbasiert: Ein Song kann aber auch anhand seiner Audio Features repräsentiert werden, z. B. Tempo, Lautstärke, Tanzbarkeit, Instrumentalität...

Aufgabe: Cluster ähnlicher Musik finden und als Empfehlung verwenden

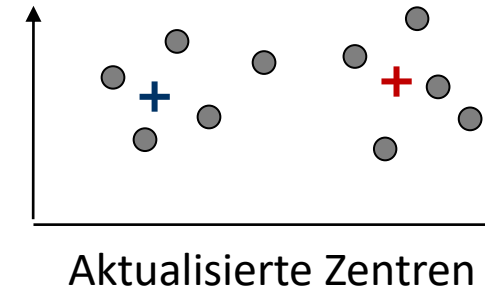
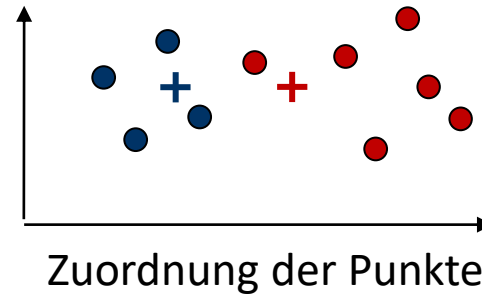
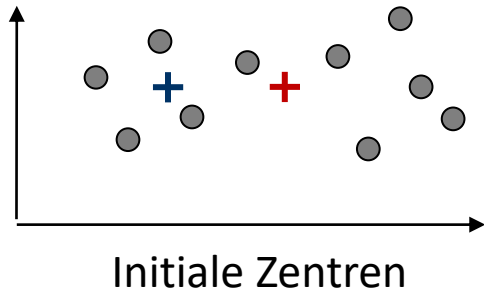


<https://towardsdatascience.com/k-means-clustering-and-pca-to-categorize-music-by-similar-audio-features-df09c93e8b64>

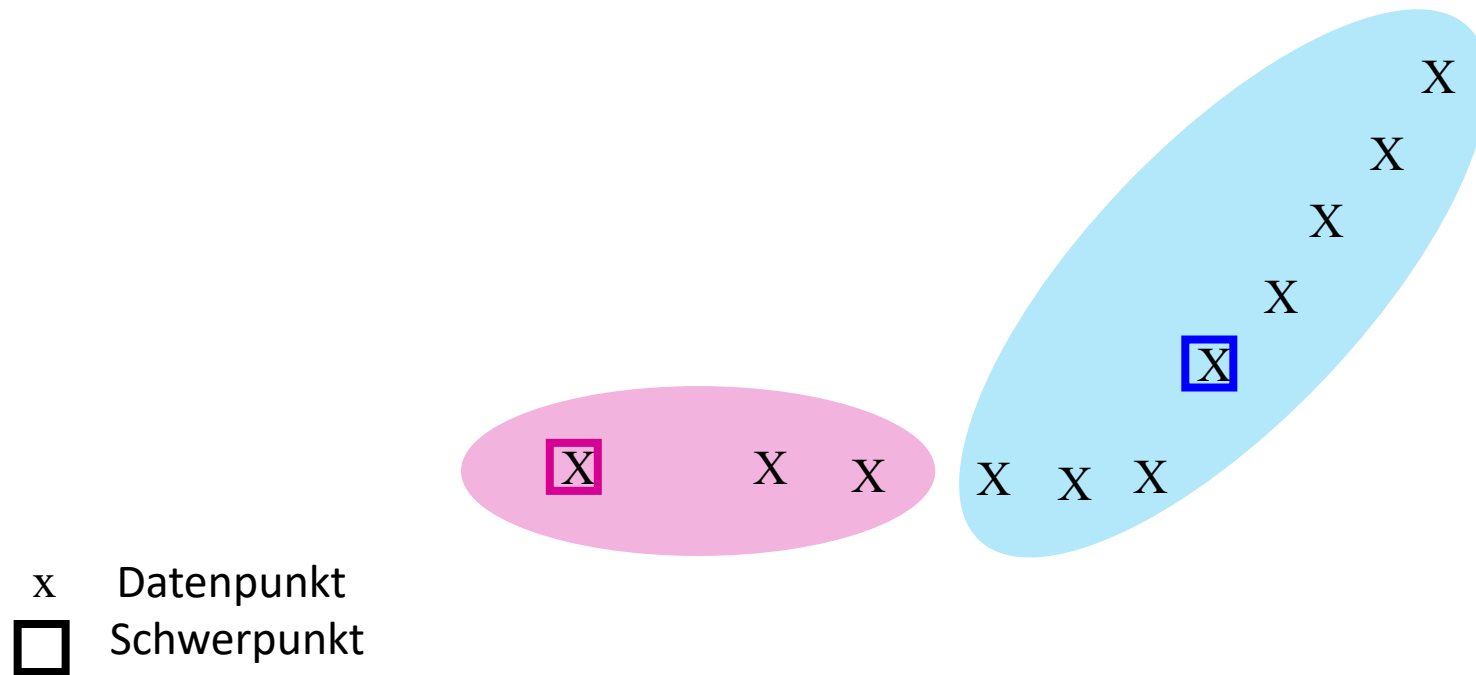
K-Means

(basic) K-means

- 1: Wähle K initiale Punkte als Zentren
- 2: **repeat**
- 3: Ordne jeden Punkt einem Cluster (Zentrum) zu
- 4: Berechne die Zentren der Cluster neu
- 5: **until** Zentren verändern sich nicht mehr

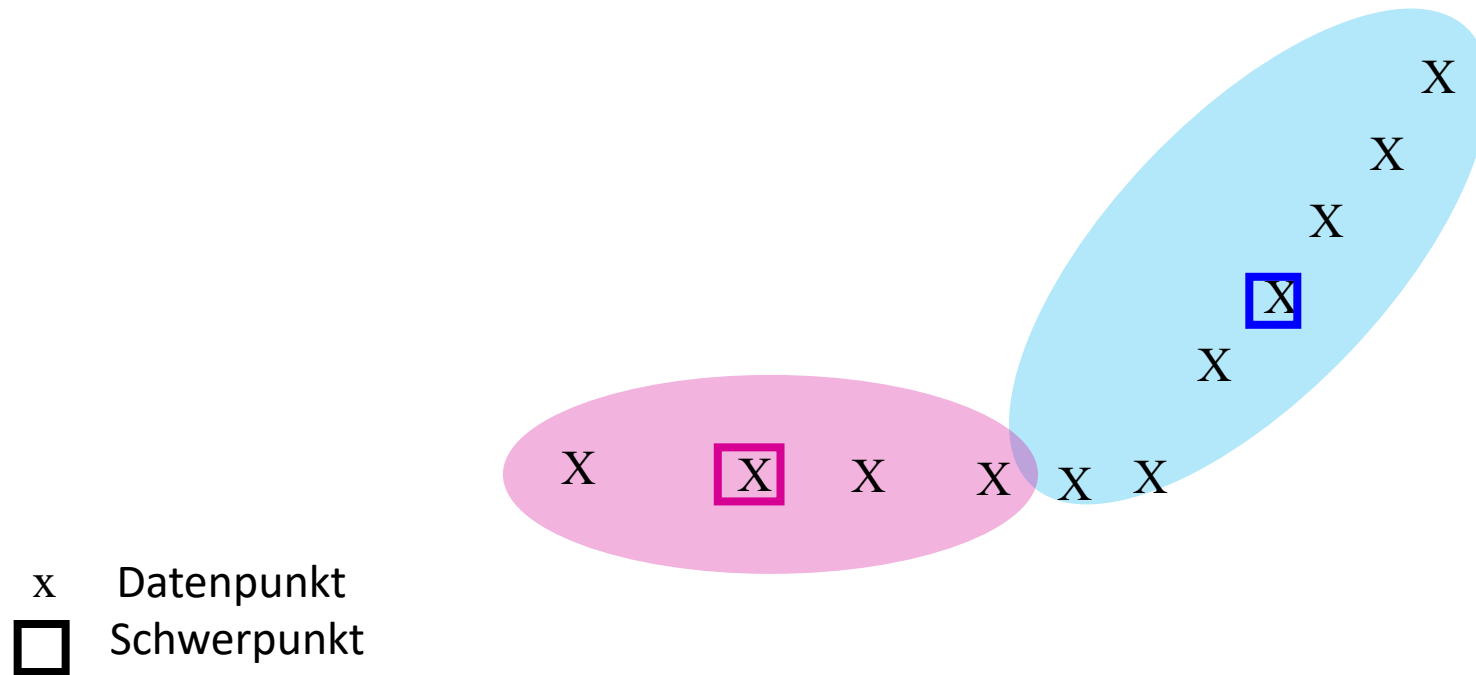


Beispiel : Cluster zuweisen



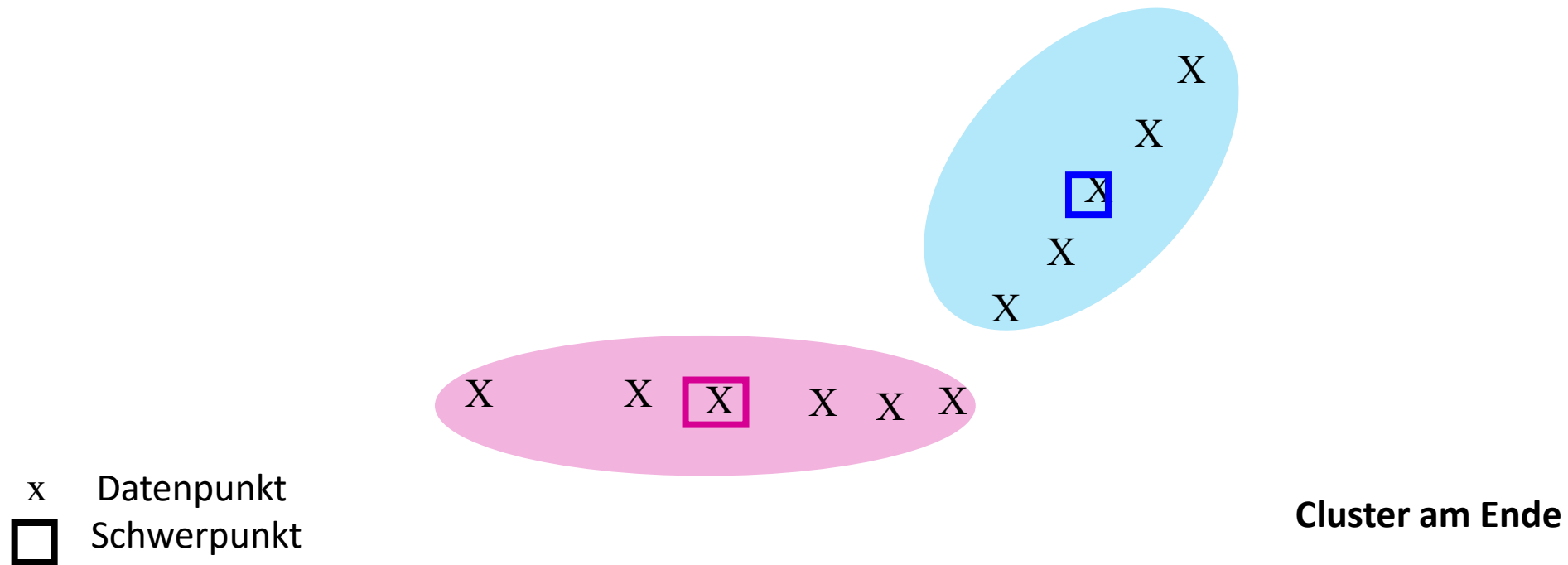
Cluster nach Runde 1

Beispiel : Cluster zuweisen



Cluster nach Runde 2

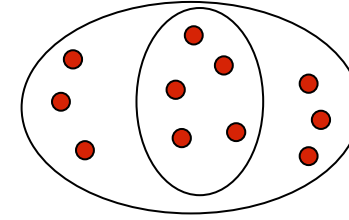
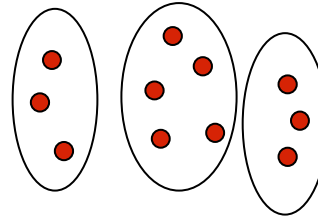
Beispiel : Cluster zuweisen



Charakteristik von Clusteralgorithmen

Clustermethoden lassen sich wie folgt charakterisieren:

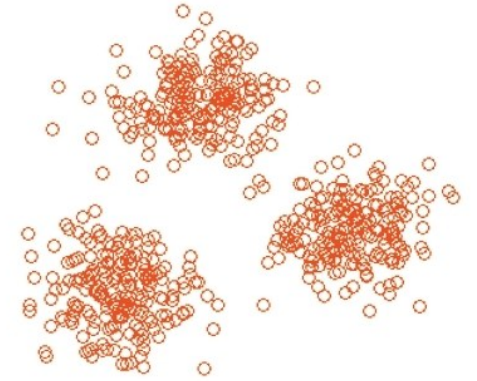
- **partitionierend vs. hierarchisch**



Methoden des Clusterings

Partitionierende Verfahren (z. B. k-means, DBSCAN):

- verwenden eine gegebene Partitionierung (Anzahl Cluster) und ein **Distanzmaß**
- Zusätzliche Gruppen können jedoch nicht gebildet werden, da die Anzahl der Cluster bereits am Anfang festgelegt wird



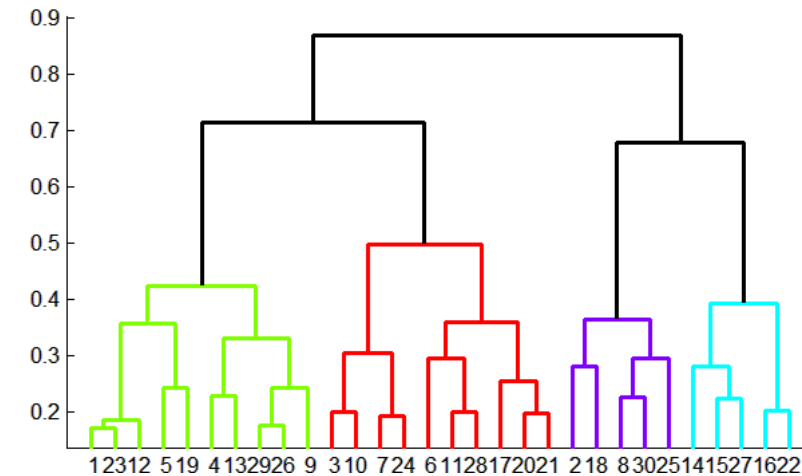
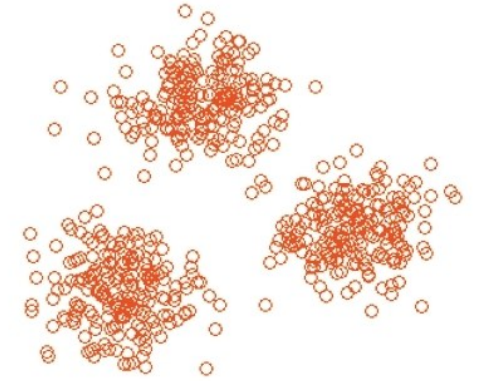
Methoden des Clusterings

Partitionierende Verfahren (z. B. k-means, DBSCAN):

- verwenden eine gegebene Partitionierung (Anzahl Cluster) und ein Distanzmaß
- Zusätzliche Gruppen können jedoch nicht gebildet werden, da die Anzahl der Cluster bereits am Anfang festgelegt wird

Hierarchisches Clustering:

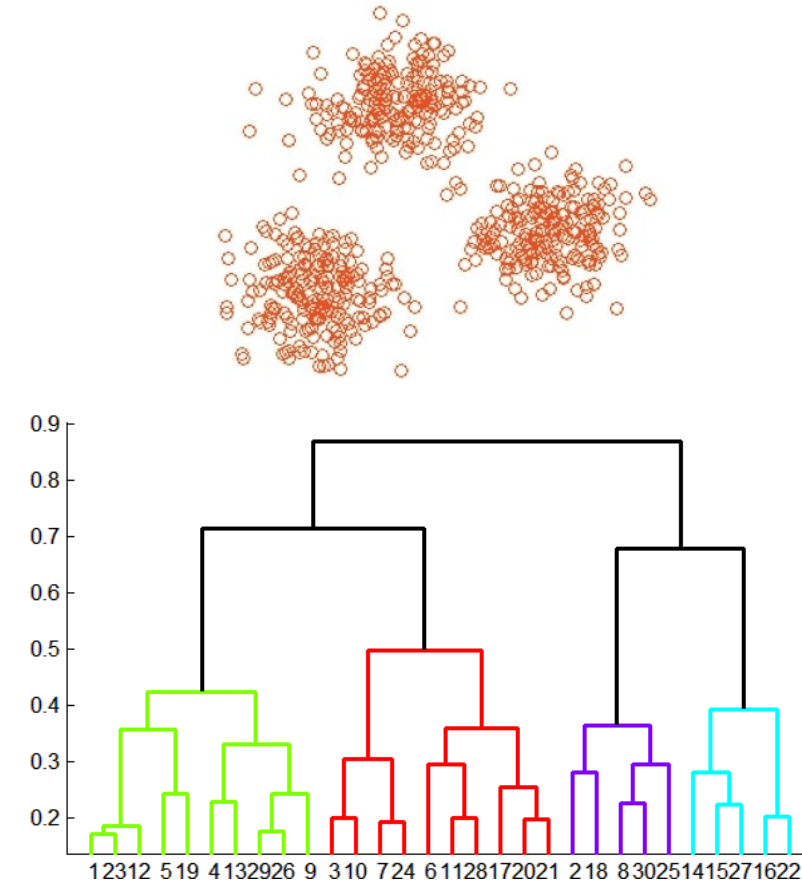
- Gruppiert ähnliche Datenpunkte basierend auf ihren Merkmalen in hierarchischen Strukturen.
- Methode um Abstand zwischen Clustern zu berechnen ist notwendig



Methoden des Clusterings

Hierarchisches Clustering:

- Es gruppiert ähnliche Datenpunkte basierend auf ihren Merkmalen in hierarchischen Strukturen.
- **Agglomerative Clustering** (bottom- up)
 - Zunächst ist jeder Punkt ein Cluster
 - Dann werden iterativ ähnliche Cluster zusammengeführt, um größere Cluster zu bilden, bis alle Datenpunkte in einem einzigen Cluster zusammengefasst sind.
- **Divisive Clustering** (top-down):
 - Beginnt mit einem Cluster das dann rekursiv aufgeteilt wird bis jeder Datenpunkt ein eigenes Cluster bildet

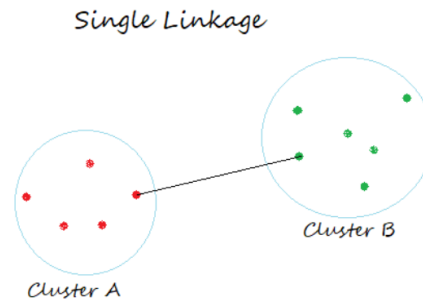


Das nächstgelegene Cluster finden- Distanzberechnung zwischen Cluster

Wie berechnet man Abstände zwischen Clustern?

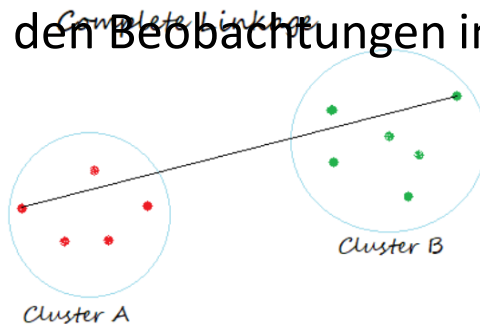
Single-Linkage:

- **Minimaler inter-cluster Abstand:** Wir berechnen alle paarweisen Abstände zwischen den Beobachtungen in Cluster A und den Beobachtungen in Cluster B und nutzen dann den kleinsten dieser Abstände als Cluster Abstand.



Complete Linkage

- **Maximaler inter-cluster Abstand:** Wir berechnen alle paarweisen Abstände zwischen den Beobachtungen in Cluster A und den Beobachtungen in Cluster B und nutzen dann den größten Abstand als Cluster Abstand.

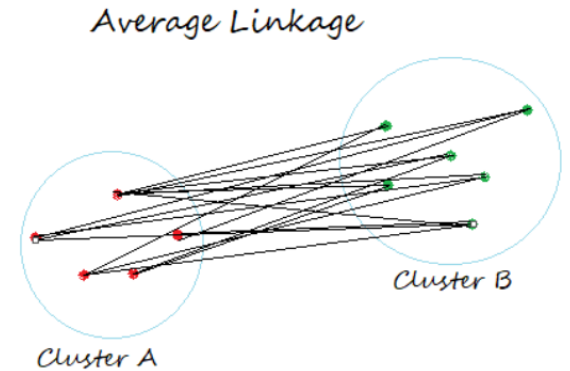


Das nächstgelegene Cluster finden- Distanzberechnung zwischen Cluster

Wie berechnet man Abstände zwischen Clustern?

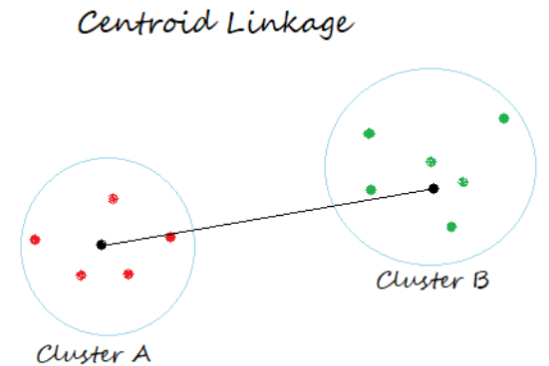
Average Linkage:

• **Durchschnittlicher inter-cluster Abstand:** Wir berechnen alle paarweisen Abstände zwischen den Beobachtungen in Cluster A und den Beobachtungen in Cluster B und nutzen dann den Durchschnitt dieser Abstände als Cluster Abstand.



Centroid Linkage:

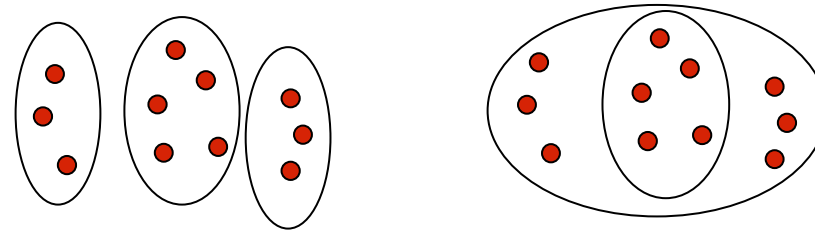
- Abstand zwischen dem Schwerpunkt für Cluster A (ein mittlerer Vektor der Länge p , wenn p die Anzahl der Features ist) und dem Schwerpunkt für Cluster B.
- Der Schwerpunkt ist ein virtueller Punkt (Vektor)



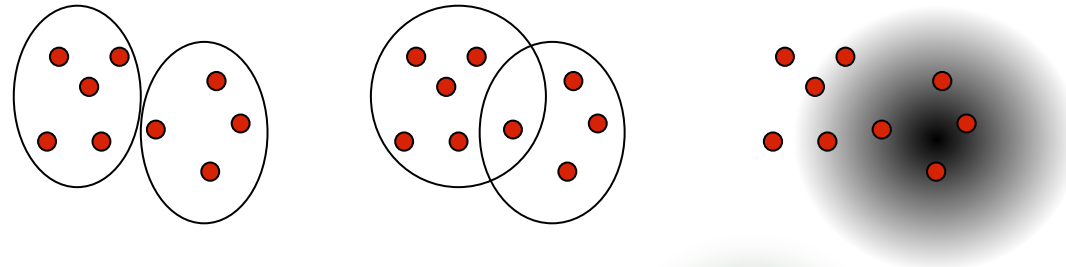
Charakteristik von Clusteralgorithmen

Clustermethoden lassen sich wie folgt charakterisieren:

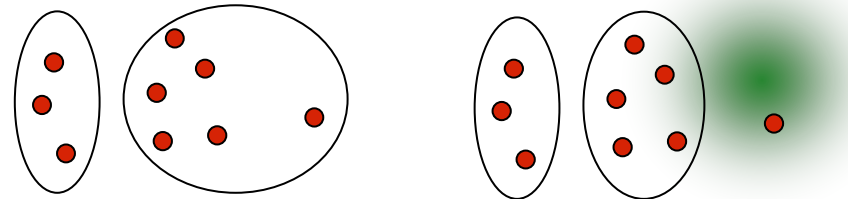
- **partitionierend vs. hierarchisch**



- **exklusiv vs. überlappend vs. fuzzy**



- **vollständig vs. partiell**



- **vorgegebene vs. dynamische Anzahl der Cluster...**

K-Means

(basic) K-means

- 1: Wähle K initiale Punkte als Zentren
 - 2: **repeat**
 - 3: Ordne jeden Punkt einem Cluster (Zentrum) zu
 - 4: Berechne die Zentren der Cluster neu
 - 5: **until** Zentren verändern sich nicht mehr
-

Charakteristik: Der Algorithmus ist

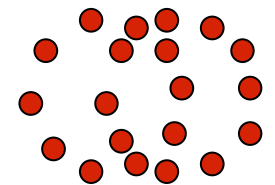
1. partitionierend
2. exklusiv
3. vollständig

und basiert auf einer vorgegebenen Anzahl von Clustern.

Eigenschaften von Clusteralgorithmen

Die Cluster selbst lassen sich wie folgt charakterisieren:

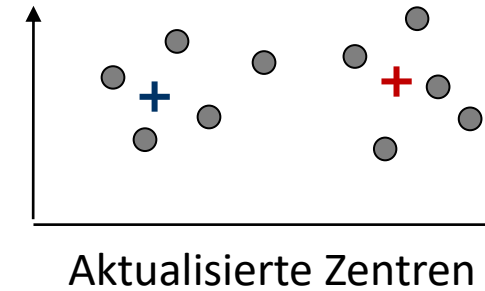
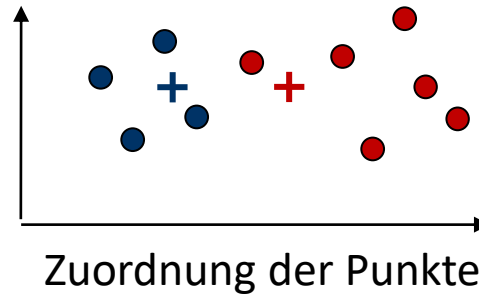
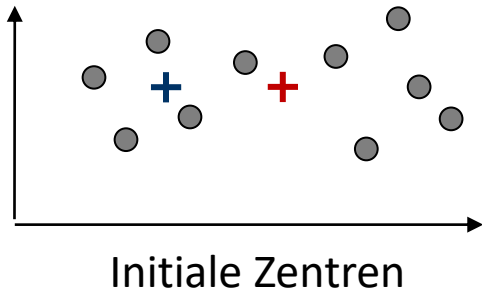
- **Zentrums-basiert:** Der Abstand zum Zentrum des Clusters ist minimal.
- **Prototyp-basiert:** Der Abstand zu einem der zuvor ausgezeichneten Prototypen ist minimal.
- **Nachbarschafts-basiert (Distanz-basiert):** Der Abstand zu mindestens einem Objekt im eigenen Cluster ist kleiner als der zu allen anderen Objekten.
- **Dichte-basiert:** Cluster sind definiert durch Regionen mit hoher Dichte, die getrennt sind durch Regionen mit geringer Dichte.
- **Modell-basiert:** Jeder Cluster entspricht einer Komponente eines probabilistischen Modells. Die Daten wurden von einer Mischung mehrerer Wahrscheinlichkeitsverteilungen erzeugt.



Designfreiheit von K-Means

(basic) K-means

- 1: Wähle K initiale Punkte als Zentren
- 2: **repeat**
- 3: Ordne jeden Punkt einem Cluster (Zentrum) zu
- 4: Berechne die Zentren der Cluster neu
- 5: **until** Zentren verändern sich nicht mehr



Welche Freiheiten haben wir bei diesem Algorithmus?

Designfreiheit K-Means

Nochmal: Welche Designfreiheit lässt uns der Algorithmus?

- Distanz- oder Ähnlichkeitsmaß (für die Zuordnung der Punkte zu den Zentren)
- Initialisierung der Zentren
- Aktualisierung der Zentren
- Abbruchkriterium

Das Maß: Typische Abstands- bzw. Distanzfunktionen sind in euklidischen Räumen in der Regel die **euklidische Distanz** im Fall von Dokumenten **die Kosinus-Ähnlichkeit**

Allgemein: Einfache Maße, um Rechenaufwand zu vermeiden.

Clustering - Zielfunktion

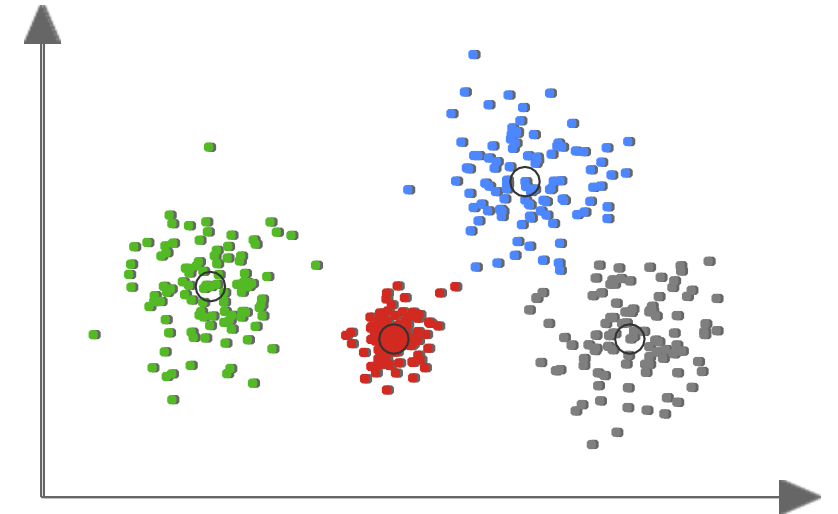
Clustering sucht nach einer Aufteilung der Daten in **ähnliche Gruppen**

- Die Datenmenge X von Beispielen (keine Zielvariable (z.B. Klassen) gegeben!)
- Parameter k zu findender Gruppen
- Abstandsmaß $d : X \times X \rightarrow R$

Ziel:

1. **Abstand innerhalb der Gruppen** soll **minimiert**,
2. **Abstand zwischen den Gruppen** soll **maximiert** werden.
 - Wird oft nicht explizit als Zielfunktion genutzt, passiert aber implizit → wenn Cluster kompakt sind ist der Inter-Clusterabstand groß
 - Ziel 2 alleine würde nicht unbedingt zu einem guten Clustering führen

➤ **Trade-off: Kompaktheit vs. Trennung**

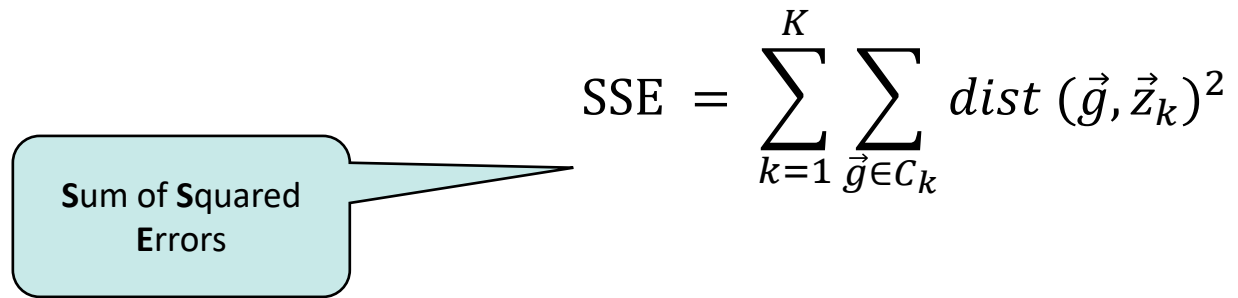


Zielfunktion

Die Berechnung der Zentren ist eng mit einer **Zielfunktion** verbunden:

Das Ziel des Clustering wird i.d.R. in einer mathematischen Funktion (**Güte- oder Kostenfunktion**) ausgedrückt.

Beispiel: Minimiere die **Summe der quadratischen Abstände jedes Objekts zum Zentrum**



A light blue callout box with a black border and a pointer towards the equation. Inside the box, the text "Sum of Squared Errors" is written in black. The equation to the right is
$$SSE = \sum_{k=1}^K \sum_{\vec{g} \in C_k} dist(\vec{g}, \vec{z}_k)^2$$

wobei $\vec{z}_k = (z_{1k}, z_{2k}, \dots, z_{dk})^T$ das Zentrum des k -ten Clusters C_k bezeichnet, d entspricht der Anzahl der Merkmale (Dimension), K der Anzahl Cluster.

Optimierung der Zielfunktion

Sobald das Abstandsmaß dist und die Zielfunktion definiert sind, können die Zentren im Sinne der Zielfunktion optimal bestimmt werden.

Dafür fassen wir die Zielfunktion als eine Funktion über den $K \cdot d$ unbekannten Komponenten z_{ik} der Zentren auf und suchen das **Minimum der Zielfunktion**.

Beispiel: SSE mit euklidischer Distanz

$$\text{SSE} = \sum_{k=1}^K \sum_{\vec{g} \in C_k} \text{dist}^2(\vec{g}, \vec{z}_k) = \sum_{k=1}^K \sum_{\vec{g} \in C_k} \sum_{i=1}^d (g_i - z_{ik})^2$$

Das entspricht der **Minimierung der Summe der Varianz innerhalb der Cluster**

Aufgabe SSE für Clustering

Gegeben seien folgende Daten:

Punkt	(x_1)	(x_2)
A	1	2
B	2	1
C	1	1
D	6	5
E	7	6
F	6	7

Eine Clustering-Zuordnung in **2 Cluster** sei bereits gegeben:

- **Cluster 1:** A, B, C
- **Cluster 2:** D, E, F

Aufgabe SSE für Clustering

1. Berechne die Zentren (Cluster-Mittelwerte) für beide Cluster.
2. Berechne für jeden Punkt:
 - a. den euklidischen Abstand zum jeweiligen Clusterzentrum
 - b. das Quadrat dieses Abstands
 - c. Berechne den SSE (Sum of Squared Errors):

$$\text{SSE} = \sum_{k=1}^K \sum_{\vec{g} \in C_k} \text{dist}^2(\vec{g}, \vec{z}_k) = \sum_{k=1}^K \sum_{\vec{g} \in C_k} \sum_{i=1}^d (g_i - z_{ik})^2$$

Optimierung der Zielfunktion

Wir leiten also den SSE nach z_{jl} ab, setzen die Ableitung auf Null und lösen die Gleichung nach z_{jl} auf:

$$\begin{aligned}
 \frac{d \text{ SSE}}{dz_{jl}} &= \frac{d}{dz_{jl}} \sum_{k=1}^K \sum_{\vec{g} \in C_k} \sum_{i=1}^d (g_i - z_{ik})^2 \\
 &= \sum_{k=1}^K \sum_{\vec{g} \in C_k} \sum_{i=1}^d \underbrace{\frac{d}{dz_{jl}} (g_i - z_{ik})^2}_{\substack{1 \\ \neq 0 \Leftrightarrow i = j \text{ und } k = l}} \quad (x_2) \\
 &= \sum_{\vec{g} \in C_l} \frac{d}{dz_{jl}} (g_j - z_{jl})^2 = \sum_{\vec{g} \in C_l} 2(g_j - z_{jl})(-1) = 0 \quad \begin{matrix} 1 \\ 5 \\ 6 \end{matrix}
 \end{aligned}$$

Auflösen nach z_{jl} ergibt:

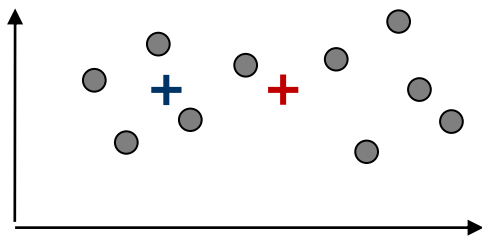
$$\sum_{\vec{g} \in C_l} 2(g_j - z_{jl})(-1) = 0 \Leftrightarrow z_{jl} = \frac{1}{|C_l|} \sum_{\vec{g} \in C_l} g_j \quad \begin{matrix} 6 \\ 7 \end{matrix}$$

Heuristische Methode für K-Means

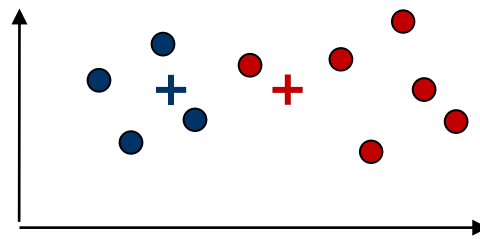
Das Problem der Minimierung der Zielfunktion wird als **NP-hart** betrachtet, weil es keine bekannte effiziente Methode gibt, um die **optimale Gruppierung für alle möglichen Datensätze in polynomialer Zeit** zu finden. Stattdessen müssen oft **heuristische** oder **approximative Methoden** (z.B. Lloyd Algorithmus) verwendet werden, um gute, aber **nicht** unbedingt **optimale Lösungen** zu erhalten.

(basic) K-means

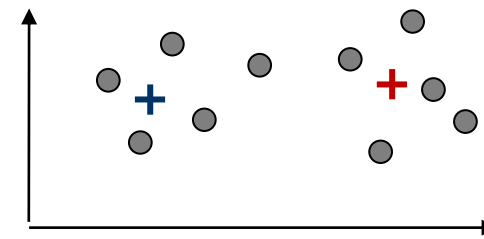
- 1: Wähle K initiale Punkte als Zentren
 - 2: **repeat**
 - 3: Ordne jeden Punkt einem Cluster (Zentrum) zu
 - 4: Berechne die Zentren der Cluster neu
 - 5: **until** Zentren verändern sich nicht mehr
-



Initiale Zentren



Zuordnung der Punkte



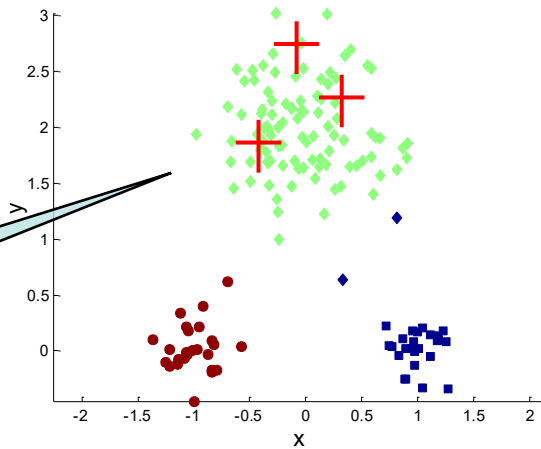
Aktualisierte Zentren

Initialisierung der Zentren

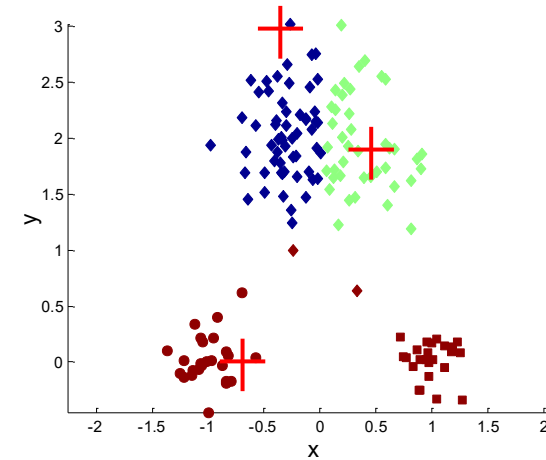
Häufiges Vorgehen: Zufällige Initialisierung via **random sampling**

Aber: Das Clusterergebnis ist **abhängig von der Initialisierung**:

Kreuze: Initiale
Zentren
Farbig: Ergebnis
des Clusterings



optimales Clustering



suboptimales Clustering

Abhilfe: **Wiederholte Anwendung** mit zufälliger Initialisierung und Entscheidung anhand der Zielfunktion.

Optimale Aktualisierung der Zentren

- In jedem Iterationsschritt des K-Means-Algorithmus werden die **Clusterzentren neu berechnet**.
- Um das SSE zu minimieren und somit eine optimale Verteilung der Zentren zu erreichen, werden die neuen Zentren auf den **Schwerpunkt** (Mittelwert) der Datenpunkte gesetzt, die dem jeweiligen Cluster zugeordnet wurden.
- Dieser Schritt ist entscheidend, um die Clusterzentren iterativ zu verschieben, bis sie stabil sind und das SSE ein lokales Minimum erreicht hat.
- Für den **SSE und die euklidische Distanz** ergeben sich **optimal aktualisierten Zentren** also dann, wenn sie auf den jeweiligen **Schwerpunkt der Cluster** gesetzt werden!

$$z_k := \frac{1}{|C_k|} \sum_{\vec{g} \in C_k} \vec{g} \text{ für } k = 1, \dots, K$$

- Jede andere Position des Clusterzentrums würde zu einem **höheren SSE-Wert** führen, da die Summe der quadrierten Abstände zu den Punkten im Cluster größer wäre.

Abbruchkriterium & Konvergenz

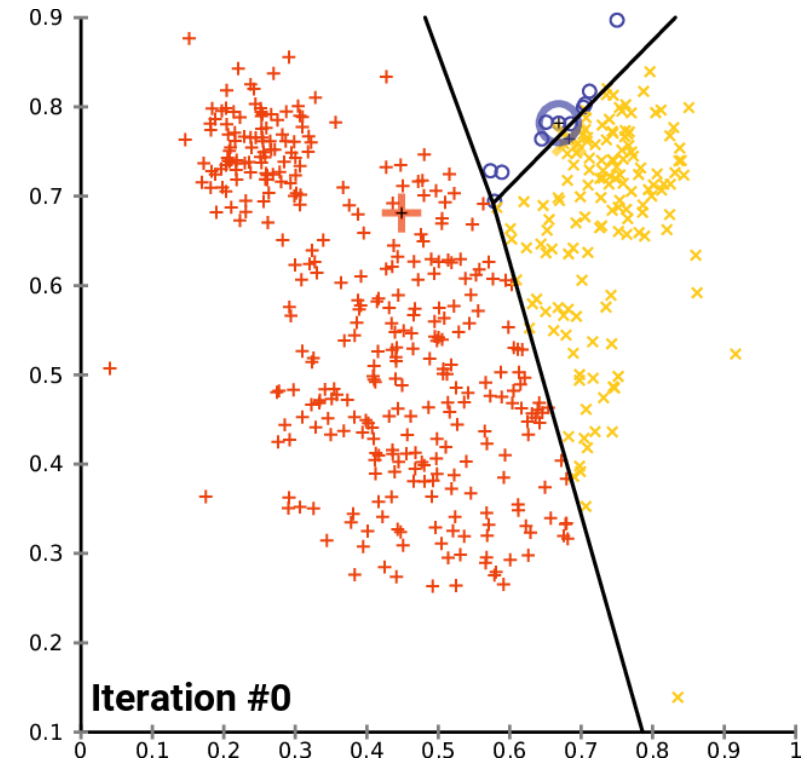
Die im Algorithmus formulierte Forderung

»Zentren verändern sich nicht mehr«

ist automatisch erfüllt, wenn **kein Wechsel zu einem anderen Cluster mehr stattgefunden hat**.

Die Forderung wird bei Bedarf (große Datenmengen) aufgeweicht, z.B. in der Form

»**maximal x% der Objekte hat das Cluster gewechselt**«.



Bildquelle: Wikipedia

Randeffekte – Leere Cluster

Bei K-Means können leere Cluster auftreten, wodurch sich die Zahl der Cluster reduziert. Abhilfe durch

Erhöhung der Clusterzahl:

Splitte das »schlechteste« (im Sinne der Zielfunktion) Cluster oder

Erzeuge ein neues Zentrum (z.B. das Objekt, das am weitesten von allen Zentren liegt).

Verringerung der Clusterzahl:

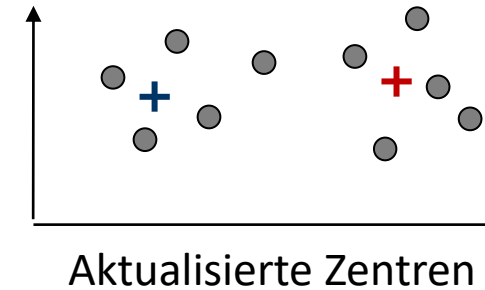
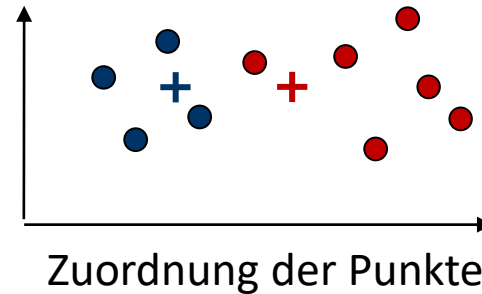
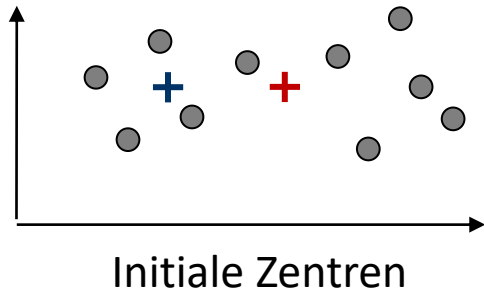
Verwerfe das Zentrum des »besten« Clusters (im Sinne der Zielfunktion) oder

Vereinige zwei Cluster (z.B. diejenigen, die zur größten Verbesserung der Zielfunktion führen).

K-Means - Komplexität?

(basic) K-means

- 1: Wähle K initiale Punkte als Zentren
- 2: **repeat**
- 3: Ordne jeden Punkt einem Cluster (Zentrum) zu
- 4: Berechne die Zentren der Cluster neu
- 5: **until** Zentren verändern sich nicht mehr



Bemerkungen

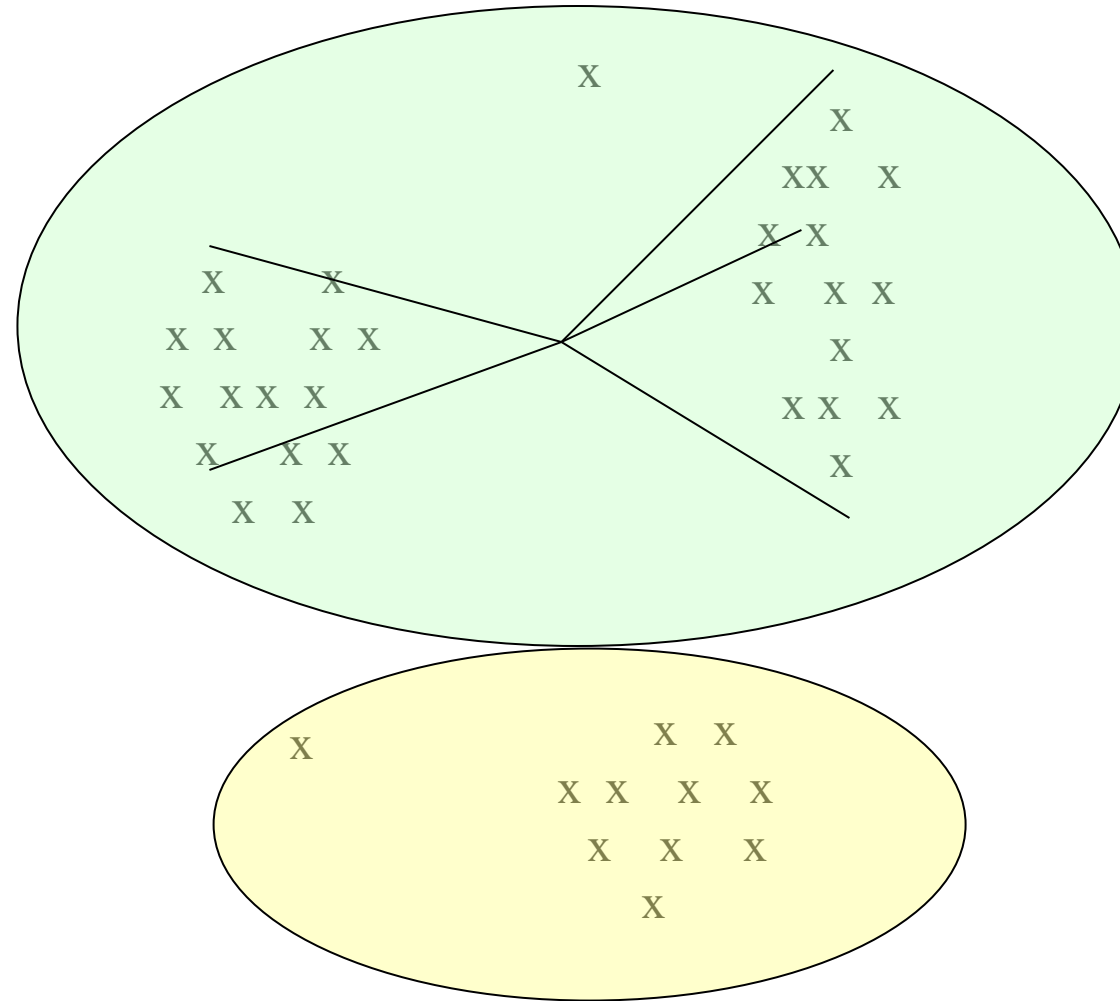
- K-Means wird (wie so oft) in unterschiedlichen Quellen unterschiedlich dargestellt: Mitunter wird die hier vorgestellte Variante als **iteriertes Minimaldistanzverfahren** bezeichnet.
- Es gibt viele Varianten und Erweiterungen des K-Means: **Median** statt Mean benutzen, Manhattan Distanz statt Euklidische Distanz nutzen
- **Achtung:** Je nach Zielfunktion (z.B. SSE) ist K-Means sehr empfindlich gegenüber **Ausreißern**!

Letzte Frage zum K-Means: **Wie bestimmen wir ein »optimales« K?**

Beispiel - Wahl der Clusteranzahl k

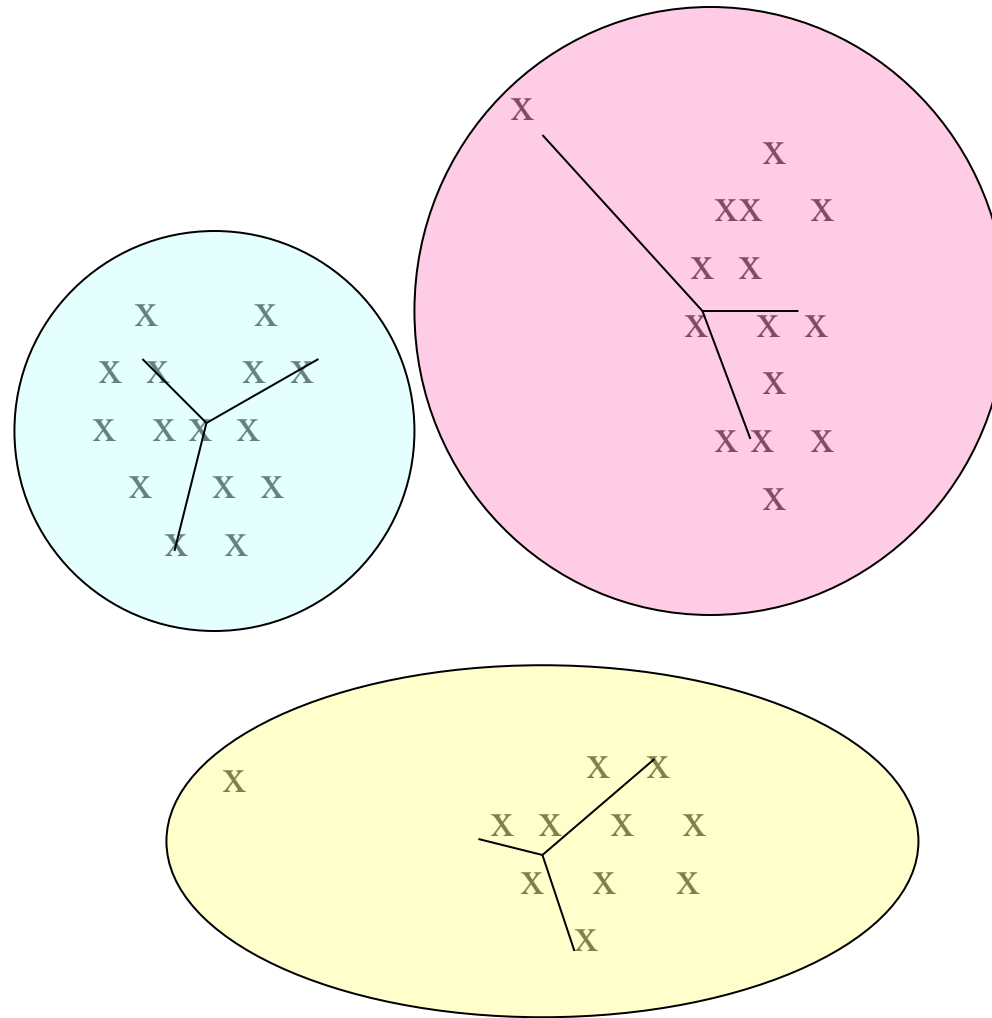
K=2 zu klein

Viele große Abstände zum
Zentrum



Beispiel - Wahl der Clusteranzahl k

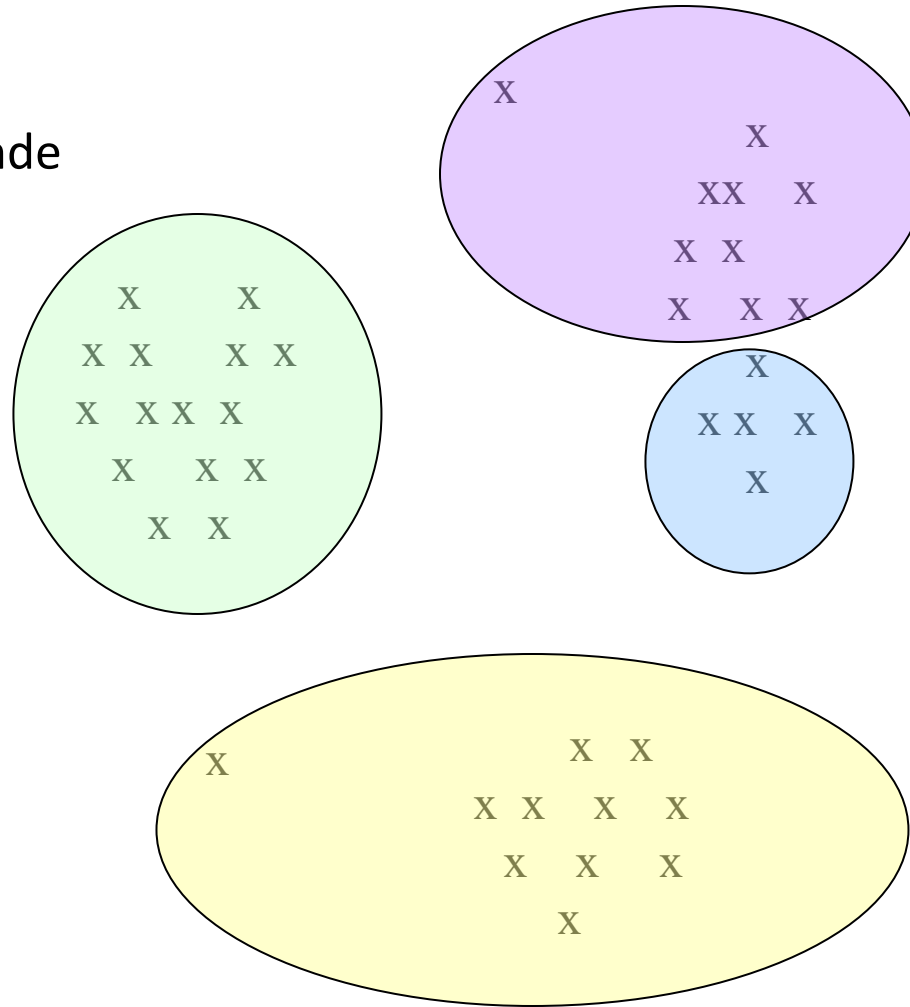
K=3 genau richtig



Beispiel - Wahl der Clusteranzahl k

K=4 zu groß

Kaum Veränderung der Abstände
zum Zentrum

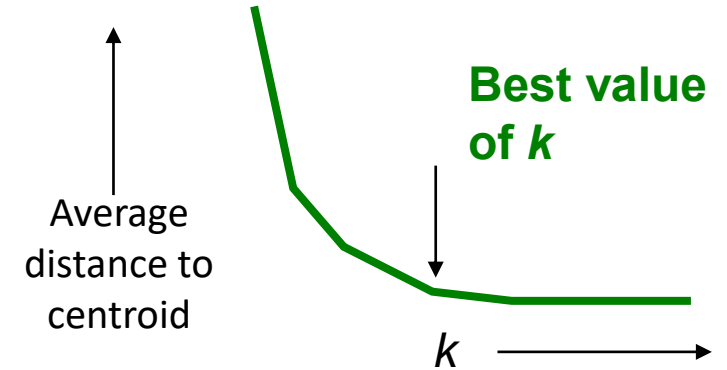


Wahl der Clusteranzahl k

Elbow Methode: Wir prüfen für verschiedene Werte von k wie sich die durchschnittliche Summe der Abstände zum Clusterzentrum (SSE oder auch Within-Cluster-Sum of Squares) verändern:

$$SSE = \sum_{k=1}^K \sum_{\vec{g} \in C_k} dist(\vec{g}, \vec{z}_k)^2$$

Man wählt den Punkt, an dem die Abnahme des SSE deutlich abflacht ("Ellenbogen").



Silhouetten-Analyse:

Berechne den durchschnittlichen Silhouetten-Score (Ähnlichkeit innerhalb jedes Clusters & Distanz zu anderen Clustern) für jedes k.

Wähle das k mit dem höchsten Score.

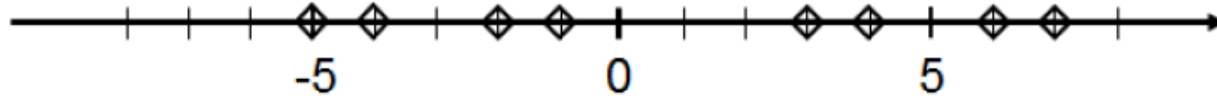
Empfehlung: Nutze mind. 2 Methoden (z. B. Elbow-Methode + Silhouetten-Score) für robuste Ergebnisse

Beispielaufgabe

Aufgabe 4 (4+2+2+2 = 10 Punkte) K-means.

Der eindimensionale Datenbestand

$$\begin{aligned}\vec{x}_1 &= (-5) & \vec{x}_2 &= (-4) & \vec{x}_3 &= (-2) & \vec{x}_4 &= (-1) \\ \vec{x}_5 &= (3) & \vec{x}_6 &= (4) & \vec{x}_7 &= (6) & \vec{x}_8 &= (7)\end{aligned}$$



soll in Cluster eingeteilt werden.

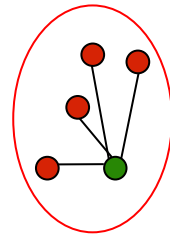
a) Wenden Sie K-means für $K = 2$ mit den (eindimensionalen) initialen Zentren $\vec{z}_1 = (-3)$ und $\vec{z}_2 = (0)$ an bis zur Konvergenz. Hinweis: Lassen Sie „unrunde“ Brüche stehen, die verschwinden nämlich im Lauf der weiteren Iterationen von selbst.

Silhouettenkoeffizient

Rousseeuw schlägt in [Rousseeuw 1987] vor, bei unbekannter Clusterzahl K den Algorithmus für $K = 2..n$ anzuwenden und schließlich das Ergebnis mit dem **besten Silhouettenkoeffizienten** zu wählen:

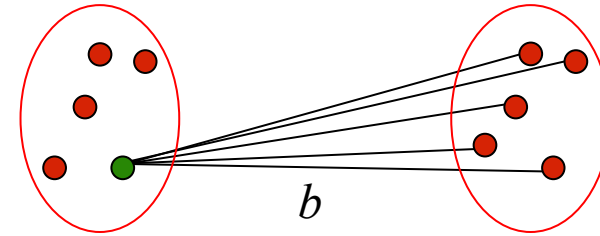
Idee: Ein gutes Ergebnis zeichnet sich durch zwei Eigenschaften aus:

Der mittlere quadratische Fehler misst nur die Nähe innerhalb der Cluster ...



Nähe zu den Objekten innerhalb der Cluster

dist klein bzgl. des eigenen Clusters



Abgrenzung zu anderen Clustern

dist groß bzgl. anderer Cluster

$$\text{dist}(\vec{x}, C_i) = \frac{1}{|C_i|} \sum_{\vec{y} \in C_i} \text{dist}(\vec{x}, \vec{y})$$

Silhouette

Die Silhouette gibt für eine Beobachtung an, **wie gut die Zuordnung zu den beiden nächstgelegenen Clustern** ist. Für jedes Objekt $\vec{x}$ wird der **Abstand zum eigenen Cluster** $C_{\vec{x}}^0$ (zum Schwerpunkt) berechnet

$$a_{\vec{x}} := \text{dist}(\vec{x}, C_{\vec{x}}^0)$$

und der **Abstand zum nächstgelegenen Cluster**

$$b_{\vec{x}} := \text{dist}(\vec{x}, C_{\vec{x}}^1)$$

berechnet.

Dann bezeichnet man

$$\text{dist}_{\vec{x}} = \frac{b_{\vec{x}} - a_{\vec{x}}}{\max\{a_{\vec{x}}, b_{\vec{x}}\}} = \begin{cases} 1 - \frac{a_{\vec{x}}}{b_{\vec{x}}} \in [0,1] & \text{für } a_{\vec{x}} \leq b_{\vec{x}} \\ \frac{b_{\vec{x}}}{a_{\vec{x}}} - 1 \in [-1,0] & \text{für } a_{\vec{x}} > b_{\vec{x}} \end{cases}$$

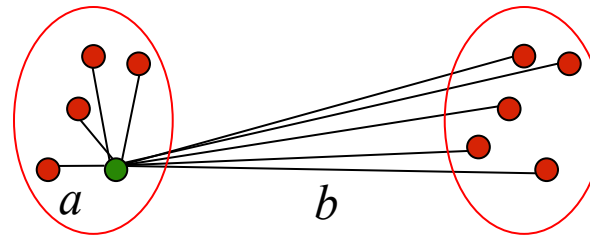
als **Silhouette** von Objekt $\vec{x}$.

Silhouettenkoeffizient

Den **Mittelwert über alle Silhouetten** eines Datenbestands bezeichnet man als **Silhouettenkoeffizient** (zu einer gegebenen Clusterung)

$$\frac{1}{N} \sum_{\vec{x} \in D} dist_{\vec{x}}$$

Der **Silhouettenkoeffizient** liegt immer im Intervall $[-1,1]$ und ein Wert **nahe der 1** indiziert ein **gutes Ergebnis** (a/b klein):

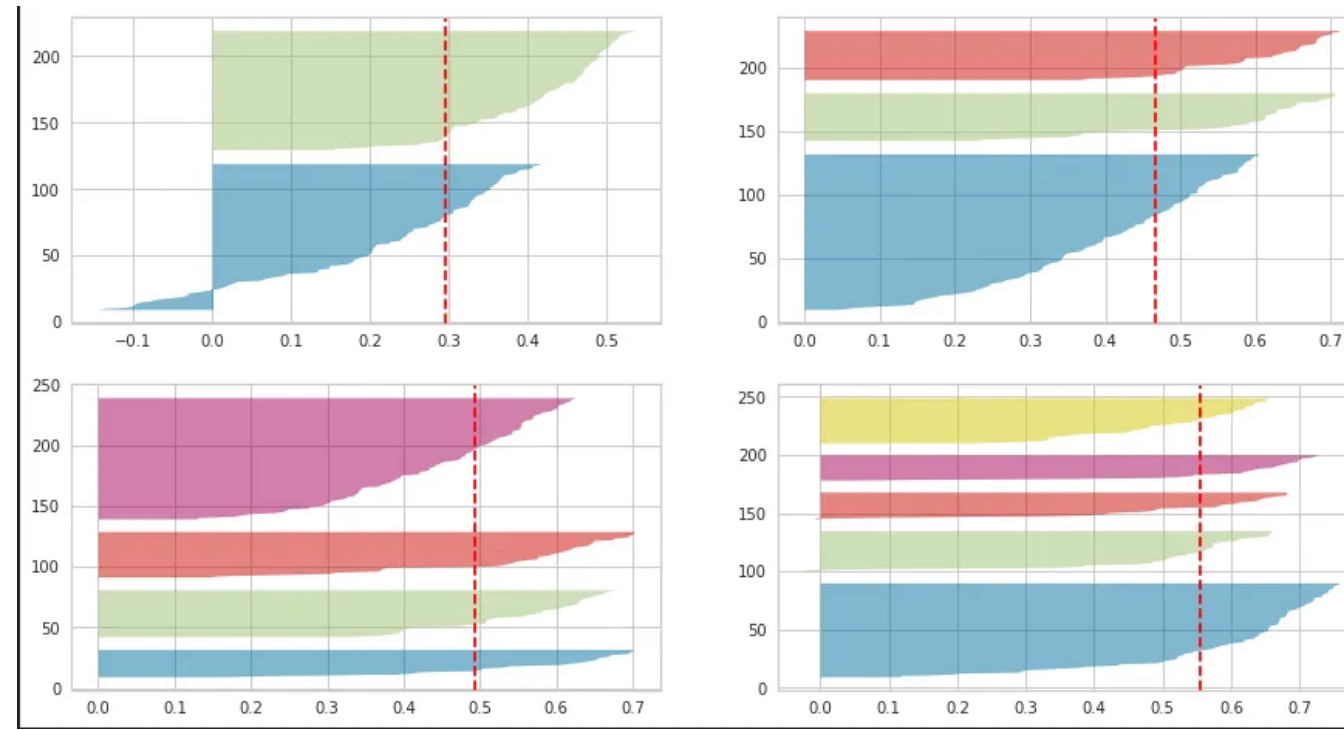


Der Silhouettenkoeffizient gilt als **weitgehend unabhängig von der Anzahl der Cluster** und eignet sich daher **zum Vergleich von Ergebnissen über unterschiedliche K hinweg**.

[Rousseeuw 1987] Peter J. Rousseeuw. *Silhouettes: A graphical aid to the interpretation and validation of cluster analysis*. In Journal of Computational and Applied Mathematics, Volume 20, Nov. 1987, pp 53-65.

Silhouettenkoeffizient - Interpretation

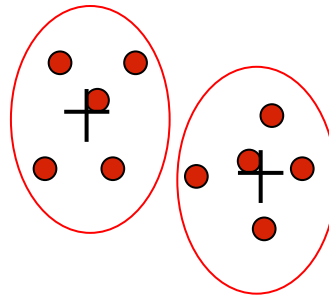
- Werte **nahe +1** bedeuten, dass der Punkt **gut zu seinem eigenen Cluster passt** und weit von anderen Clustern entfernt ist (gute Trennung).
- Werte **um 0** deuten darauf hin, dass der Punkt an der Grenze **zwischen zwei Clustern** liegt.
- **Werte unter 0** bedeuten, dass der Punkt wahrscheinlich dem **falschen Cluster** zugeordnet wurde
- **Silhouette Plot:**
 - Die Höhe der Cluster-Balken zeigt die **Clustergröße**.
 - Die Länge der Balken entspricht dem **Silhouette Wert** des jeweiligen Punkts.
 - Eine vertikale gestrichelte Linie markiert den durchschnittlichen Silhouette-Wert aller Punkte (Gesamtqualität der Clusterung)



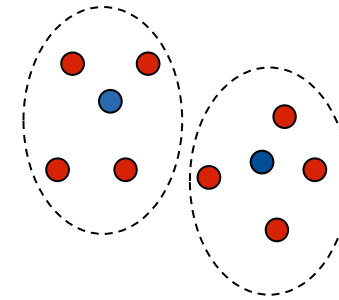
K-Means Varianten

Der K-Means-Algorithmus basiert auf der Berechnung von **Zentren**, die in der Regel durch den **Mittelwert der Objekte eines jeden Clusters** gegeben sind.

In manchen Fällen interessiert aber weniger das Cluster selber als viel mehr die Zentren der Cluster selbst (als **Repräsentant, Prototyp**):



Zentrums-basiert

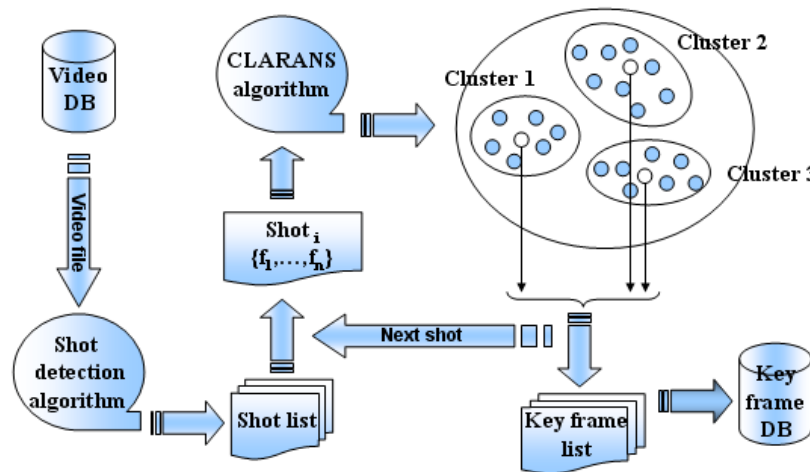


Prototyp-basiert

Beispiel: Repräsentative Standbilder in Videos

Beispiel: Für **Videos** (Fernsehfilm, Nachrichten, Videoclips) sollen repräsentative Standbilder ermittelt werden (zur Indexierung, Zusammenfassung, etc.)

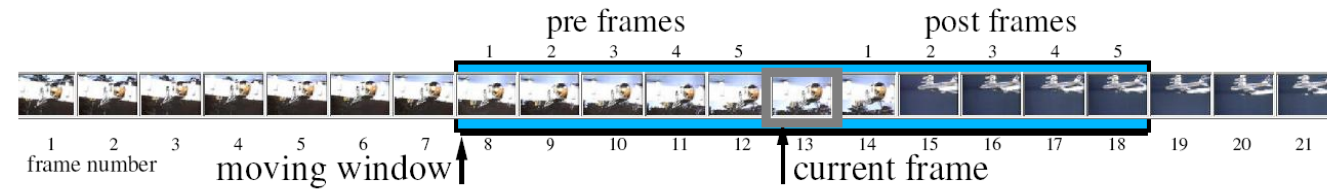
Dazu werden Videos in der Regel zunächst in einzelne Sequenzen zerlegt, die jeweils einer **Kameraeinstellung (shot)** entsprechen. Aus jedem dieser shots sollen dann ein oder mehrere **Prototypen** ausgewählt werden, die für die Sequenz repräsentativ sind.



Quelle: Y. Hadi, F. Essannouni and R.O.H. Thami. *Unsupervised Clustering by k-medoids for Video Summarization*. 2006

Shot Detection

Die Schnitte zwischen den einzelnen Kameraeinstellungen werden z.B. durch die Berechnung von Unähnlichkeiten innerhalb eines gleitenden Fensters detektiert.



Nun sollen noch innerhalb der shots K **repräsentative Standbilder** ermittelt werden, d.h. die Bilder einer jeden Kameraeinstellung werden **Prototyp-basiert geclustert** und die Prototypen der Cluster als Repräsentanten (**keyframes**) verwendet.

Quelle: S. M. M. Tahaghoghi, H. E. Williams, J. A. Thom and T. Volkmer. *Video Cut Detection using Frame Windows*. 2005.

Der Medoid eines Clusters

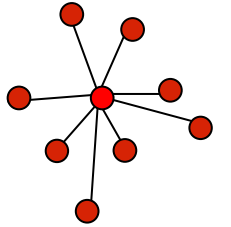
Typische Vertreter von **prototypbasierten Cluster-Algorithmen** definieren die Zentren nicht als mittleren Datenpunkt, sondern als **Medoiden**.

Ein Medoid ist ein Datenpunkt innerhalb eines Clusters, der die **geringste durchschnittliche Distanz zu allen anderen Punkten** in diesem Cluster hat.

Der Medoid ist (auch bei euklidischer Distanz) wesentlich **robuster gegenüber Ausreißern** als der Schwerpunkt.

Unter der Voraussetzung, dass ein geeignetes Distanzmaß $dist$ vorliegt, ist der **Medoid** $\vec{m}_k$ des k -ten Clusters C_k definiert durch

$$\operatorname{argmin}_{\vec{y} \in C_k} \sum_{\vec{x} \in C_k} dist(\vec{x}, \vec{y})$$



K-Medoid = PAM – Partitioning Around Medoids

Initialisierung (BUILD-Phase):

- Wähle k Objekte als erste Medoids aus.
- Oft wird das Objekt mit der geringsten Gesamtdistanz zu allen anderen als erstes Medoid gewählt.

Optimierung (SWAP-Phase):

- Versuche, die aktuelle Auswahl der Medoids zu verbessern, indem du einen Medoid mit einem nicht ausgewählten Punkt tauschst und prüfst, ob sich dadurch die Gesamtdistanz verringert.
- Dieser Schritt wird iterativ wiederholt, bis keine Verbesserung mehr möglich ist.

Zuweisung:

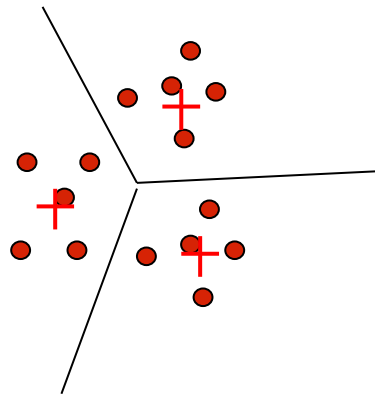
- Weise jedes Objekt dem nächstgelegenen Medoid zu, um die Cluster zu bilden.

PAM - Vor- und Nachteile

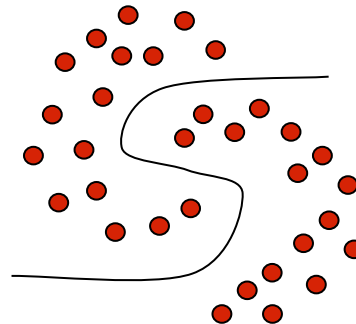
- **Vorteil:** PAM ist robuster gegenüber Ausreißern als k-means, da Medoids tatsächliche Datenpunkte sind und nicht durch Extremwerte verzerrt werden.
- **Nachteile:**
 - Die Anzahl der Cluster k muss vorab festgelegt werden.
 - Nicht für große Datenmengen geeignet da Laufzeitkomplexität quadratisch

Eigenschaften K-Means & Varianten

- **Anfälligkeit** gegenüber **Rauschen** und **Ausreißern** - alle Objekte gehen ein in die Berechnung des Zentroids ein
- Die **Anzahl K der Cluster** ist oft schwer zu bestimmen
- Starke **Abhängigkeit** (Ergebnis und Laufzeit) von **initialen Clusterzentren**
- Sowohl der K-means als auch die genannten K-medoid-Algorithmen **maximieren die Kompaktheit der Cluster** im Sinne der gewählten Distanz/Ähnlichkeit.
- Cluster müssen **konvexe Form** haben (Voronoi-Mosaik - kugelförmige Form), d.h. **linear trennbar sein**
- Die Behandlung nicht-konvexer Cluster wird dadurch unmöglich



linear trennbare Cluster

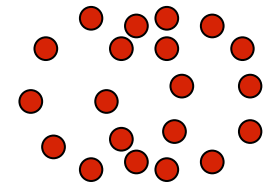


nichtlinear trennbare Cluster

Eigenschaften von Clusteralgorithmen

Die Cluster selbst lassen sich wie folgt charakterisieren:

- Zentrums-basiert: Der Abstand zum Zentrum des Clusters ist minimal.
- Prototyp-basiert: Der Abstand zu einem der zuvor ausgezeichneten Prototypen ist minimal.
- Nachbarschafts-basiert: Der Abstand zu mindestens einem Objekt im eigenen Cluster ist kleiner als der zu allen anderen Objekten.
- **Dichte-basiert:** Cluster sind definiert durch Regionen mit hoher Dichte, die getrennt sind durch Regionen mit geringer Dichte.
- Konzeptuelle Cluster: Objekte eines Clusters teilen eine gemeinsame Eigenschaft, die sich aus dem gesamten Datenbestand ergibt.



DBSCAN

DBSCAN – Density-Based algorithm for discovering clusters in large spatial databases with noise.

DBSCAN wurde 1996 von M. Ester, H.-P. Kriegel (Universität München), J. Sander und H. Xu veröffentlicht

<http://www.dbs.informatik.uni-muenchen.de/cms/Research>

Die Idee ist motiviert durch die Detektion von Ausreißern:

- **Beobachtung:** Ausreißer zeichnen sich durch **Vereinzelung** aus.
- Folglich lässt sich ein »**Nicht-Ausreißer**« dadurch charakterisieren, dass in der Umgebung **eine Mindestanzahl an anderen Objekten liegt**, d.h. die **Dichte der Objekte in seiner Umgebung soll eine gewisse Schwelle überschreiten**.

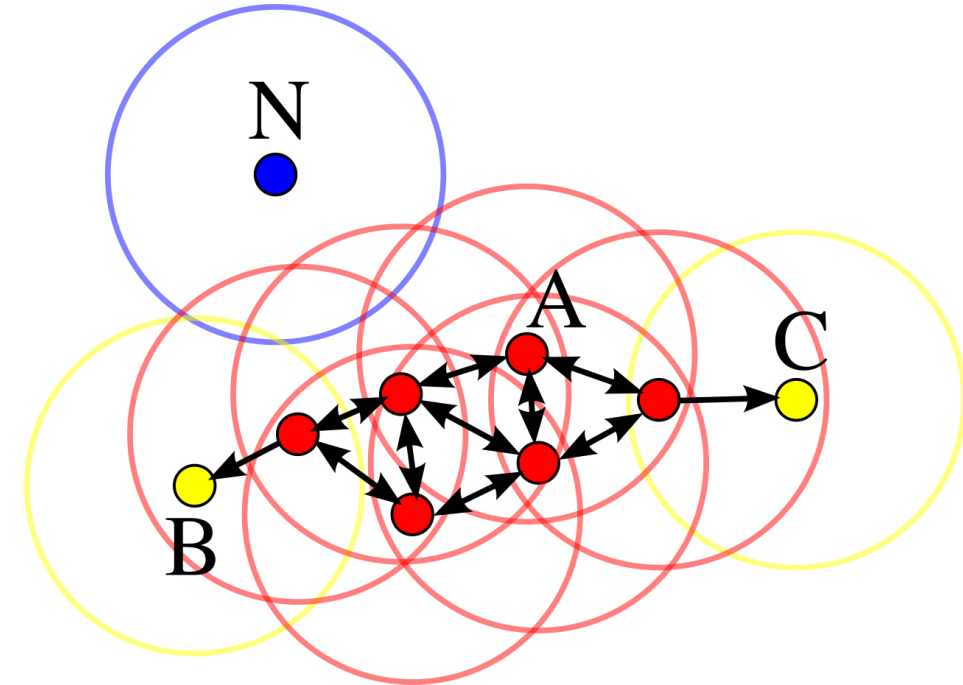
DBSCAN - Grundlagen

Die Grundidee des Algorithmus ist der Begriff der **Dichteverbundenheit**. Zwei Objekte gelten als **dichte-verbunden**, wenn es **eine Kette von dichten Objekten** gibt, die diese Punkte miteinander verbinden.

Die durch **dieselben Kernobjekte** miteinander **verbundenen Objekte bilden ein Cluster**. Objekte, die nicht Teil eines dichte-verbundenen Clusters sind, werden als **Rauschen** bezeichnet.

In DBSCAN gibt es drei Arten von Punkten:

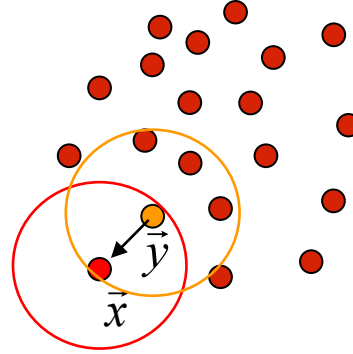
- **Kernobjekte** (A), welche selbst **dicht** sind.
- **Rand-Objekte** (B & C): Dies sind Objekte, die zwar von einem Kernobjekt des Clusters erreicht werden können, selbst aber nicht dicht sind (dichte-erreichbare Objekte). Anschaulich bilden diese den Rand eines Clusters.
- **Rauschpunkte** (N), die weder dicht noch dichte-erreichbar sind (also solche Punkte die in keiner Nachbarschaft liegen) .



Bildquelle: wikipedia

Direkte Erreichbarkeit

In der ε -Umgebung von $\vec{x}$ befindet sich ein Objekt $\vec{y}$, in dessen Umgebung die Mindestanzahl nicht unterschritten wird:



Definition Direkte Erreichbarkeit: Ein Objekt $\vec{x}$ ist **direkt erreichbar** von Objekt $\vec{y}$, wenn

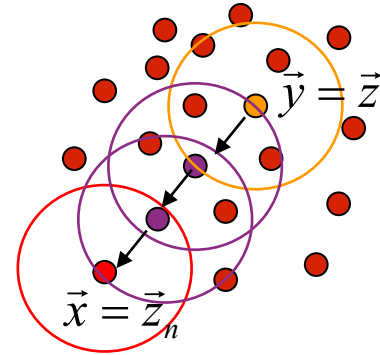
1. $\vec{x} \in U_\varepsilon(\vec{y})$ ($\Leftrightarrow \vec{y} \in U_\varepsilon(\vec{x})$) und
2. $|U_\varepsilon(\vec{y})| > \text{MinPts}$ ($\Leftrightarrow \vec{y}$ ist **Kernobjekt**)

Erreichbarkeit

Definition Erreichbarkeit: Ein Objekt $\vec{x}$ ist **erreichbar** von Objekt $\vec{y}$, wenn eine **Kette von Objekten**

$$\vec{y} = \vec{z}_1, \vec{z}_2, \dots, \vec{z}_n = \vec{x}$$

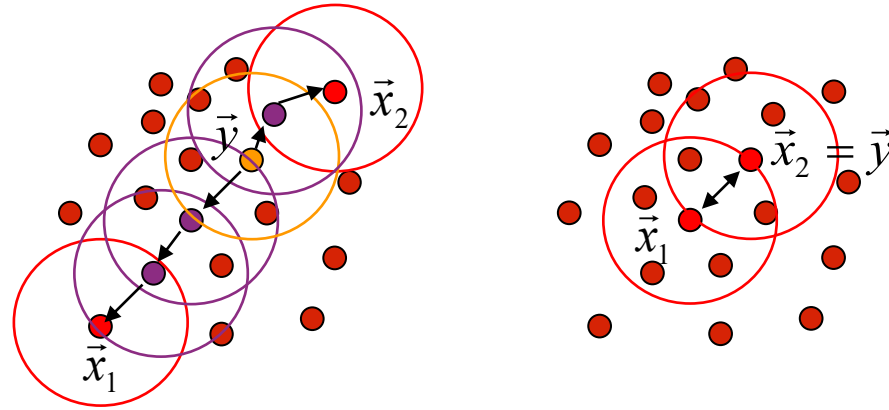
existiert, sodass $\vec{z}_{i+1}$ von $\vec{z}_i$ direkt erreichbar ist.



Nur das letzte Objekt der Kette kann Randobjekt sein, d.h. nur bei dem letzten Objekt können weniger als `MinPts` Objekte in der Umgebung liegen!

Verbundenheit

Definition Verbindung: Ein Objekt $\vec{x}_1$ ist mit einem Objekt $\vec{x}_2$ **verbunden**, wenn es ein Objekt $\vec{y}$ gibt, sodass $\vec{x}_1$ und $\vec{x}_2$ von $\vec{y}$ erreichbar sind.



Cluster (im Sinne von DBSCAN)

Intuitive Definition von Clustern:

- Maximale Mengen von verbundenen Objekten.

Intuitive Definition von Ausreißern:

- Objekte, die zu keinem Cluster gehören.

Formale Definition Cluster: Ein **Cluster** C ist eine Teilmenge einer Datenbasis **mit mindestens einem Kernobjekt**, die folgende Bedingungen erfüllt:

- Ein Cluster ist **maximal**, wenn er alle Punkte enthält, die **dichteverbunden** sind (d. h., sie liegen innerhalb eines Radius ϵ um einen Kernpunkt und erfüllen die MinPts-Bedingung).
 - Also: Liegt $\vec{y}$ in C und ist $\vec{x}$ erreichbar von $\vec{y}$, dann liegt auch $\vec{x}$ in C .
 - Liegen $\vec{x}$ und $\vec{y}$ in C , so sind sie verbunden.

Definition: Ausreißer sind genau diejenigen Objekte, die zu keinem Cluster gehören.

Eigenschaften DBSCAN

DBSCAN

```
DBSCAN(DB, distFunc, eps, minPts) {
    C := 0
    for each point P in database DB {
        if label(P) ≠ undefined then continue
    Loop */
        Neighbors N := RangeQuery(DB, distFunc, P, eps)
        if |N| < minPts then {
            label(P) := Noise
            continue
        }
        C := C + 1
        label(P) := C
        SeedSet S := N \ {P}
        for each point Q in S {
            if label(Q) = Noise then label(Q) := C
            if label(Q) ≠ undefined then continue
        border point) */
            label(Q) := C
            Neighbors N := RangeQuery(DB, distFunc, Q, eps)
            if |N| ≥ minPts then {
                S := S ∪ N
            }
        point) */
    }
}

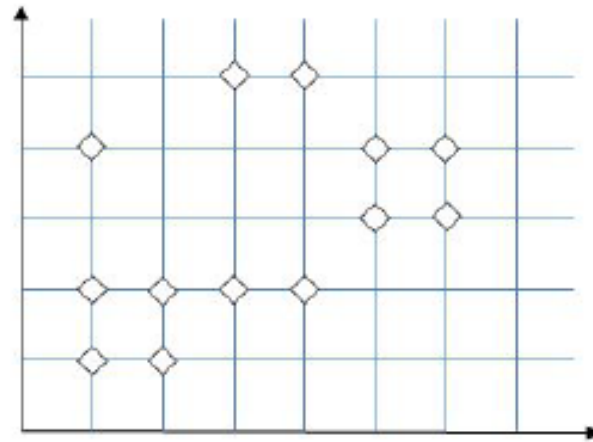
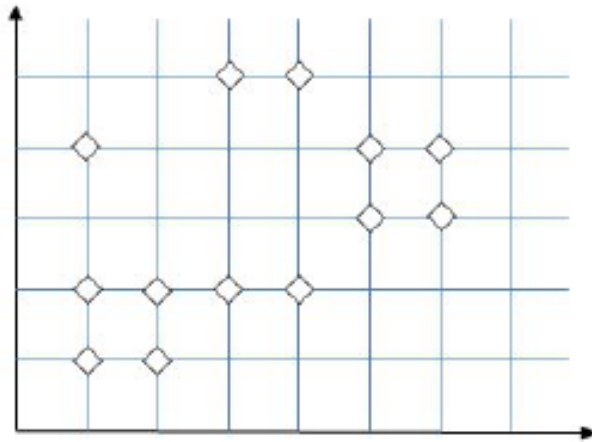
/* Cluster counter */
/* Previously processed in inner
Loop */
/* Find neighbors */
/* Density check */
/* Label as Noise */
/* next cluster label */
/* Label initial point */
/* Neighbors to expand */
/* Process every seed point Q */
/* Change Noise to border point */
/* Previously processed (e.g.,
border point) */
/* Label neighbor */
/* Find neighbors */
/* Density check (if Q is a core
point) */
/* Add new neighbors to seed set
*/
```

```
RangeQuery(DB, distFunc, Q, eps) {
    Neighbors N := empty list
    for each point P in database DB {
        if distFunc(Q, P) ≤ eps then
            N := N ∪ {P}
    }
    return N
}
```

Übung DBSCAN

Aufgabe 4 (4+2+2+4+2 = 14 Punkte) Cluster-Algorithmen.

a) Skizzieren Sie (durch Einkreisen), welche Cluster entstehen, wenn der DBSCAN mit $\text{MinPts}=2$ (d.h. bei Kernpunkten liegen mindestens zwei weitere Punkte in der jeweiligen ε -Umgebung), der euklidischen Distanz und mit $\varepsilon = 1.1$ (links) und $\varepsilon = 1.5$ (rechts) angewandt wird. Markieren Sie Ausreißer farblich.



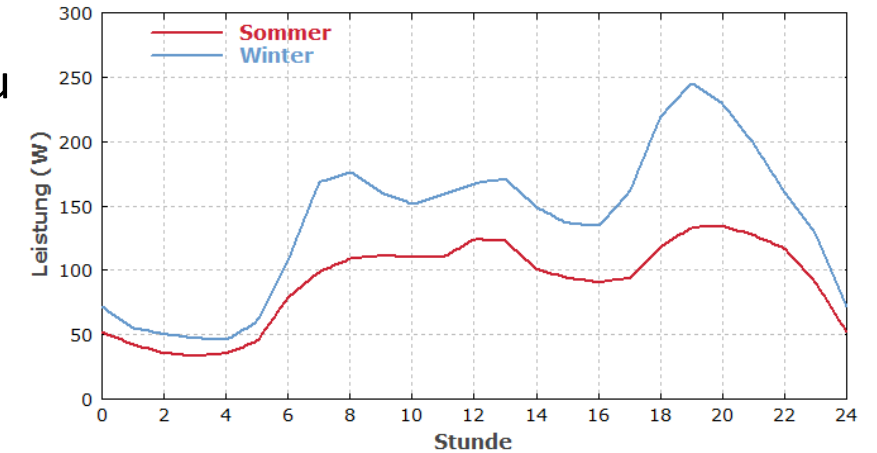
Vergleich von Clusteralgorithmen

Eigenschaft	DBSCAN	k-Means	Hierarchical Clustering
Clusterform	Beliebig	Konvex	Beliebig
Clusteranzahl	Automatisch	Vorab festgelegt (k)	Automatisch (Dendrogramm)
Rauschen	Ja	Nein	Nein
Skalierbarkeit	Gut (mit Indexstrukturen)	Gut	Schlecht
Parameter	ϵ (Eps), MinPts	k	Linkage-Methode, Distanz
Überlappung	Nein	Nein	Ja

Clustering auf Zeitreihen - Anwendungsbeispiele

- **Energie & Versorgungsunternehmen:**

- Lastmanagement: Versorger gruppieren Verbraucher nach ihren Stromverbrauchsmustern (Smart Meter Daten), um Lastspitzen zu identifizieren und Tarife für das Nachfragemanagement zu gestalten.
- Verbraucherprofile: Erstellung typischer Verhaltensprofile für verschiedene Kundengruppen (z. B. Industrie vs. Privathaushalte) zur besseren Netzplanung.

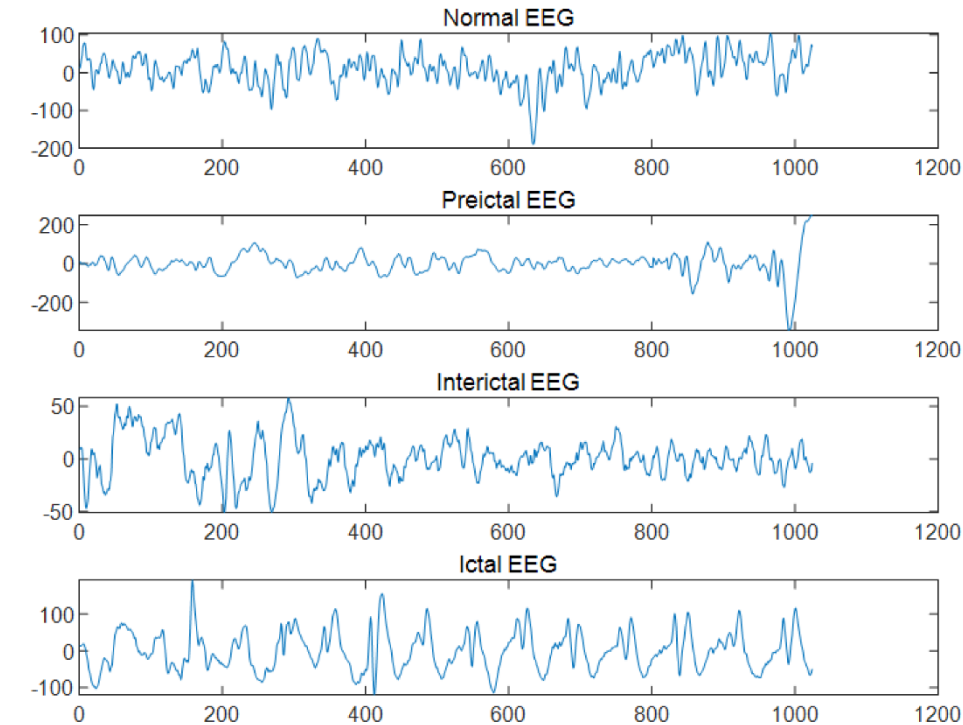


- **Finanzwesen:**

- Portfolio-Optimierung: Identifizierung von Aktien oder Finanzinstrumenten mit ähnlichen Kursbewegungen zur Risikominimierung und Diversifizierung.
- Anomalieerkennung: Detektion ungewöhnlicher Handelsaktivitäten, die auf Betrug oder Marktmanipulation hindeuten könnten.

Clustering auf Zeitreihen - Anwendungsbeispiele

- **Gesundheitswesen:** Gruppierung von Patienten mit ähnlichen Trends bei Vitalzeichen (z. B. Blutdruck, EKG) zur Personalisierung von Behandlungsplänen.
- **Klima & Umwelt:**
 - Emissionsmuster: Clustering von Luftschadstoff-Zeitreihen (z. B. Feinstaub oder Stickoxide) hilft dabei, räumlich homogene Belastungszonen in Städten zu identifizieren und die Effektivität von Umweltzonen zu prüfen.
 - Pegelstandsanalyse: Flüsse werden nach ihrem saisonalen Abflussverhalten (Regime) gruppiert, um Hochwasserrisiken besser einzuschätzen oder die Wasserverfügbarkeit für die Landwirtschaft zu planen.



Clustering (Clusteranalyse) auf Zeitreihen

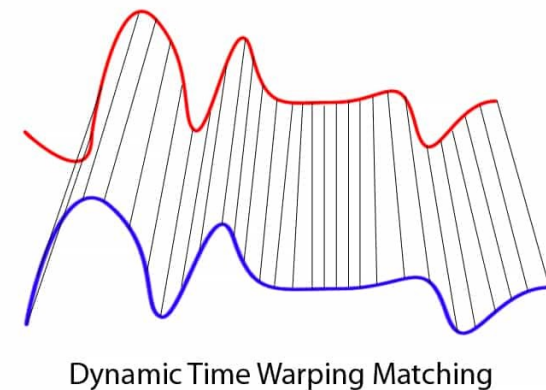
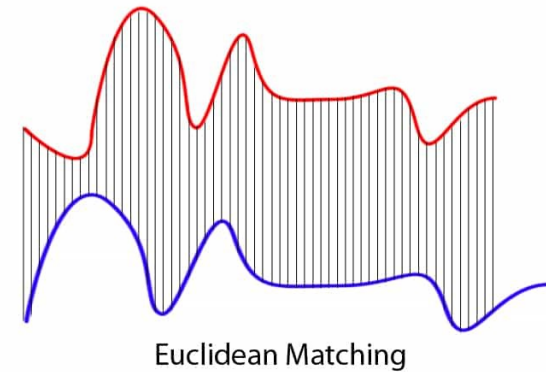
Es können die bekannten Clusterverfahren verwendet werden.

Bei gleich langen Zeitreihen wird meist die **Euklidische Distanz** oder andere übliche Distanzmaße genutzt.

Problem: die euklidische Distanz und andere Minkowski Normen, berechnen die Distanz mit einer fixen Zeitachse und vergleichen 2 Zeitreihen in dem die beiden Werte der Zeitreihen zu einem bestimmten Zeitpunkt betrachten.

Lösungen:

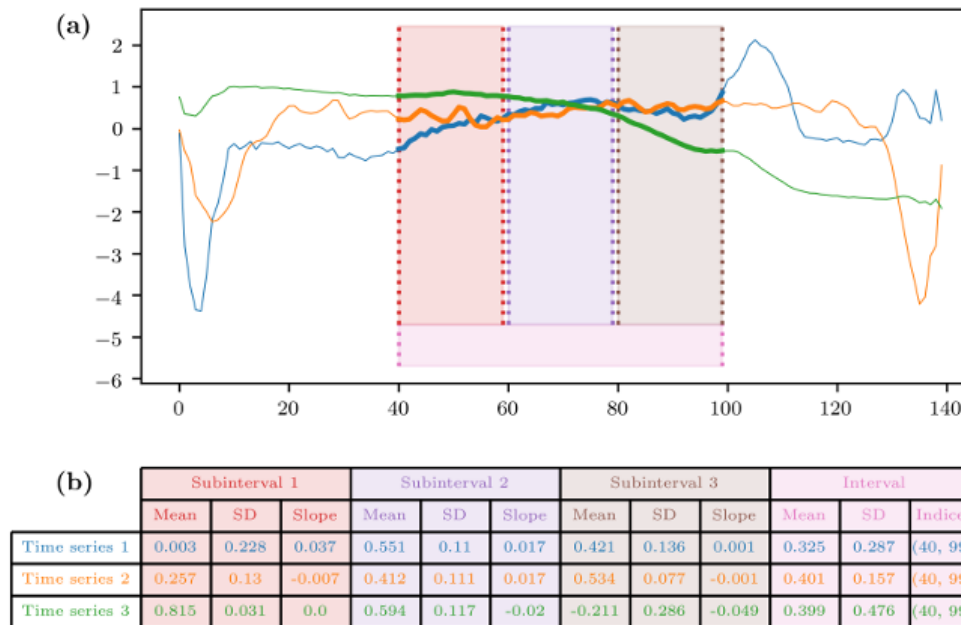
- **Geeignete Distanzmaße verwenden** → DTW oder andere wie Shape-based Distanzen: z. B. Time Warp Edit Distance (TWE), Move-Split-Merge (MSM).
- **Feature aus den Zeitreihen extrahieren** (nicht die Originalzeitreihe verwenden)



Clustering (Clusteranalyse) auf Zeitreihen

Feature aus den Zeitreihen extrahieren (nicht die Originalzeitreihe verwenden)

- Feature werden aus der Zeitreihe extrahiert.
- Wir unterteilen die Zeitreihen in äquidistante Intervalle und verwenden Aggregationsfunktionen wie Mittelwert, Varianz, Minimum, Maximum oder andere Funktionen, um Features zu generieren
- Vorteile: Reduziert Dimension, verbessert Cluster-Qualität (durch Aggregation/Glätten bei Ausreißern/Lücken)

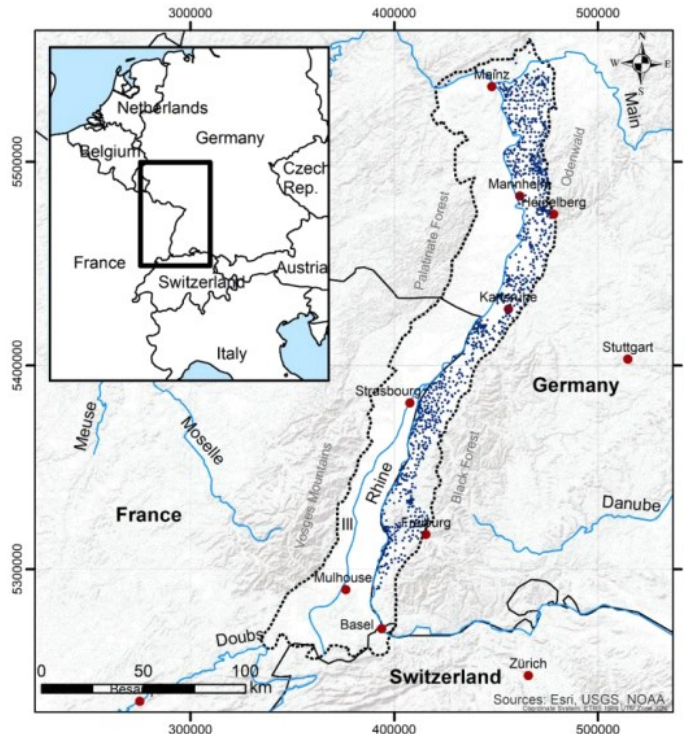


Clustering auf Zeitreihen

Beispiel: Datensatzes von ca. 1200 Grundwasserstands-Zeitreihen in BW und Hessen in wöchentlicher Auflösung über mehrere Jahre (1986-2016)

Aufgrund von üblicherweise heterogener Datenqualität in Datensätzen von Umweltzeitreihen (Zeitreihenlänge, Sampling Intervall, Datenlücken) wurde ein **feature-basierter Ansatz** gewählt

- Standardmaße der deskriptiven Statistik und Features die die Dynamik der Zeitreihen beschreiben soll
- z.B. Position des Maximums im Kalenderjahr, Median, jährliche Varianz, lang abfallende Grundwasserstände, maximale Änderung des Mittelwertes von zwei aufeinanderfolgenden Jahren, Periodizität, Verteilungsschiefe



Wunsch et. al 2021: Feature-basiertes Clustering von Umweltzeitreihen mit Self-Organizing-Map-Ensembles, Informatik 2020

Clustering auf Zeitreihen

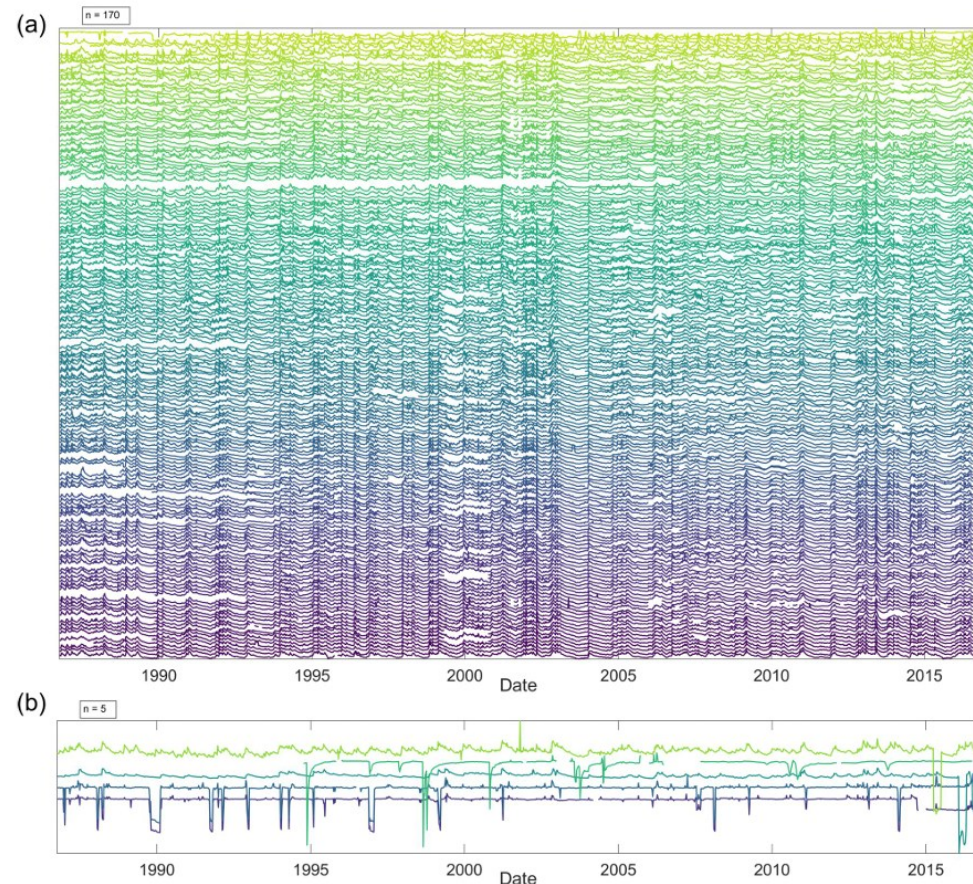


Abb. 1: Standardisierte, gestapelte Zeitreihen der Cluster 3 (a) charakterisiert durch stark korrelierte Verläufe sowie Cluster 21 (b) charakterisiert durch Ausreißer und Inhomogenitäten in der Zeitreihe.

Ergebnis:

das Clustering zeigt **generelle Dynamik-Unterschiede** zwischen dem **nördlichen und mittleren bzw. südlichen Oberrheingraben**.

Durch die reine räumliche Verteilung der Cluster, aber auch durch die Korrelation der Cluster mit räumlichen Daten zu wichtigen Einflussfaktoren, können die Cluster zudem hydrogeologisch interpretiert werden → Fachliche Interpretation

Es gibt Cluster mit:

- geringer Variabilität die mit räumlichen Bereichen übereinstimmen in denen viel Grundwasser entnommen wird und wenig Grundwasser neugebildet wird (nördlicher Bereich)
- Inhomogenitäten und Ausreißern
- Mehr Variabilität und stärkerer Periodizität in Bereichen mit höherer Grundwasserneubildung und weniger intensiver Grundwassernutzung (mittleren und südlichen Oberrheingraben)

Clusteralgorithmen für Zeitreihen

Anpassung klassischer Algorithmen:

- k-Means für Zeitreihen: Verwendung von DTW als Distanzmaß (DTW-k-Means).
- **k-Shape**: Clusterzentren als “Durchschnitts-Zeitreihen”.
- Hierarchisches Clustering: Agglomerativ oder divisiv mit DTW/anderen Metriken.
- Dichtebasierte Methoden: DBSCAN mit zeitreihenspezifischen Distanzen (z. B. DTW-DBSCAN).

Zusammenfassung Clustering

- Clustering Algorithmen gehören zu **unsupervised learning**, man unterscheidet Cluster-Algorithmen nach den ihren Eigenschaften:
 - Partitionierend vs. hierarchisch
 - Exklusiv vs. überlappend
 - Vollständig vs. partiell
 - Vorgegebene Anzahl k vs. Dynamisch
 - Zentrumsbasiert vs. Prototypen-basiert vs. Dichte-basiert....
- Ein häufig benutztes Maß, um Cluster-Algorithmen zu bewerten ist der **Silhouette Koeffizient**
- [DBSCAN Tutorial](#)
- Für Zeitreihen sollten in der Regel spezielle Distanzmaße wie DTW genutzt werden oder Feature mit Hilfe von Aggregationen erstellt werden

